

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN ĐIỆN TỬ



BÁO CÁO ĐỒ ÁN THỰC TẬP KỸ SƯ
ĐỀ TÀI: THIẾT KẾ LỖI IP TĂNG TỐC CHO BỘ
SOFTMAX ATTENTION TRÊN KIT KV260
GVHD: TS. TRƯỜNG QUANG VINH

HỌ VÀ TÊN	MÃ SỐ SINH VIÊN
HOÀNG TIẾN PHÁT	2212506

TP. HỒ CHÍ MINH

MỤC LỤC

MỤC LỤC HÌNH ẢNH	3
MỤC LỤC BẢNG	4
I. TỔNG QUAN.....	5
1. Giới thiệu đề tài	5
2. Kiến trúc hệ thống.....	6
3. Bài nghiên cứu liên quan.....	8
II. KIẾN TRÚC PHẦN CỨNG	9
1. Cấu trúc IP AXI Linear	9
1.1. Tổng quan module wrapper	9
1.2. Tổng quan kiến trúc core linear.sv	10
1.3. Tổng quan FSM IP AXI Linear	18
2. Cấu trúc IP AXI Softmax	19
2.1. Tổng quan module wrapper	19
2.2. Tổng quan kiến trúc core softmax.sv	20
2.3. FSM IP Softmax	22
III. MÔ PHỎNG TRÊN VIVADO.....	24
1. Cấu trúc thực thi	24
2. Cấu trúc mô phỏng.....	25
3. Kết quả mô phỏng	28
IV. THỰC HIỆN TRÊN PHẦN CỨNG FPGA KV260.....	29
1. Cấu trúc thực thi	29
2. Memory mapping	31
3. Thông số thiết lập	32
4. Kết quả thực thi với Ubuntu Linux	33
5. So sánh tài nguyên sử dụng.....	35
V. TÀI LIỆU THAM KHẢO.....	36
VII. PHỤ LỤC.....	37
1. Phụ lục A – Mã nguồn dự án	37
2. Phụ lục B – Report Utilization	37

MỤC LỤC HÌNH ẢNH

Hình 1: Sơ đồ kiến trúc bus hệ thống.....	7
Hình 2: Sơ đồ khối module wrapper ip_axi_linear.....	9
Hình 3: Module ip_axi_linear.....	10
Hình 4: Sơ đồ khối của file linear.sv.....	12
Hình 5: Sơ đồ khối chức năng của khối DEMUX 1 to 64 (K data).....	13
Hình 6: Sơ đồ khối chức năng của khối Ping-Pong buffer Query data.....	14
Hình 7: Sơ đồ khối chức năng của khối module matmul_ip	15
Hình 8: Sơ đồ khối chức năng của khối Ping-pong result data	16
Hình 9: Sơ đồ khối chức năng của khối pe_unit.....	17
Hình 10: Sơ đồ FSM Linear dạng Moore.....	18
Hình 11: Sơ đồ khối module wrapper ip_axi_softmax	19
Hình 12: module ip_axi_softmax.....	20
Hình 13: Sơ đồ khối file softmax.sv	21
Hình 14: FSM IP Softmax dạng Moore	22
Hình 15: Block Design mô phỏng Vivado	24
Hình 16: Sơ đồ khối thực hiện testbench	26
Hình 17: Kết quả tcl của tb và giá trị rtl softmax được lưu trong mem file	28
Hình 18: Softmax data được push vào BRAM DST từ DMA S2MM.....	28
Hình 19: Cấu trúc Block Design.....	29
Hình 20: Sơ đồ khối Code C.....	30
Hình 21: Sơ đồ khối luồng thực thi bằng Baremetal.....	31
Hình 22: Sơ đồ khối luồng thực thi bằng linux thông qua SD card.....	32
Hình 23: Kết quả thực thi dự án bằng linux với Q[64x64] và K[64x64]	33
Hình 24: Kết quả thực thi dự án bằng linux với Q[128x64] và K[64x64]	34
Hình 25: Kết quả thực thi dự án bằng linux với Q[256x64] và K[64x64]	34
Hình 26: Design Timing Summary	37
Hình 27: Power summary	37
Hình 28: Design Runs Summary	38

MỤC LỤC BẢNG

Table 1: Thông tin của các IP	7
Table 2: Các module trong IP AXI Linear	11
Table 3: Các khối chức năng trong file linear.sv	12
Table 4: Các khối chức năng của khối Ping-Pong buffer Query data	14
Table 5: Các khối chức năng của khối module matmul_ip	16
Table 6: Các khối chức năng của khối Ping-pong result data	17
Table 7: Các khối chức năng của khối module pe_unit	18
Table 8: Bảng chức năng FSM của file linear.sv	19
Table 9: Các khối chức năng của file softmax.sv	22
Table 10: Bảng chức năng FSM của file softmax.sv	22
Table 11: Thông tin các IP được sử dụng trong testbench	25
Table 12: Địa chỉ các IP được sử dụng trong testbench	25
Table 13: Thông số chuẩn hóa khi mô phỏng testbench	28
Table 14: Thông tin các IP được sử dụng khi thực thi trên KV260	29
Table 15: Địa chỉ các IP được sử dụng khi thực thi trên KV260	31
Table 16: Thông số chuẩn hóa được sử dụng khi thực thi trên KV260	32
Table 17: Bảng so sánh tài nguyên	35

I. TỔNG QUAN

Hệ thống tính Softmax($Q \times K^T$), trên phần cứng Zynq UltraScale+ (KV260), điều khiển bằng bare-metal C qua Vitis hoặc Ubuntu Linux. Cấu trúc 2 tầng:

- IP Linear: tính $S = Q \times K^T$.
- IP Softmax: nhận S, tính softmax score theo từng hàng, trả về attention weight (Q1.15 unsigned).
- Giao tiếp luồng dữ liệu xuyên suốt: DMA stream dữ liệu K, Q vào IP Linear, IP Linear tính toán và stream trực tiếp kết quả sang IP Softmax, IP Softmax xuất kết quả cuối cùng về lại DMA
- Điều khiển: mỗi IP có control-plane AXI4-Lite riêng, data-plane sử dụng AXI4-Stream.

1. Giới thiệu đề tài

Trong bối cảnh trí tuệ nhân tạo ngày càng phát triển, các mô hình học sâu dựa trên kiến trúc Transformer đang dần trở thành tiêu chuẩn vàng trong hàng loạt ứng dụng, từ xử lý ngôn ngữ tự nhiên đến thị giác máy tính. Trái tim của cấu trúc Transformer chính là cơ chế Self-Attention, đóng vai trò quan trọng trong việc nắm bắt các phụ thuộc dài hạn của dữ liệu. Mặc dù mang lại hiệu suất mô hình vượt trội, cơ chế Attention lại đòi hỏi khối lượng tính toán khổng lồ và tiêu tốn nhiều băng thông bộ nhớ. Quá trình này bao gồm phép nhân ma trận $S = Q \times K^T$, kết hợp với hàm chuẩn hóa phi tuyến tính Softmax để tạo ra ma trận trọng số chú ý. Tính phức tạp của phép toán phân thức, hàm mũ và yêu cầu truy cập bộ nhớ liên tục đã tạo ra rào cản lớn khi triển khai Transformer lên các Edge Devices – nơi có những giới hạn khắt khe về năng lượng và tài nguyên phần cứng.

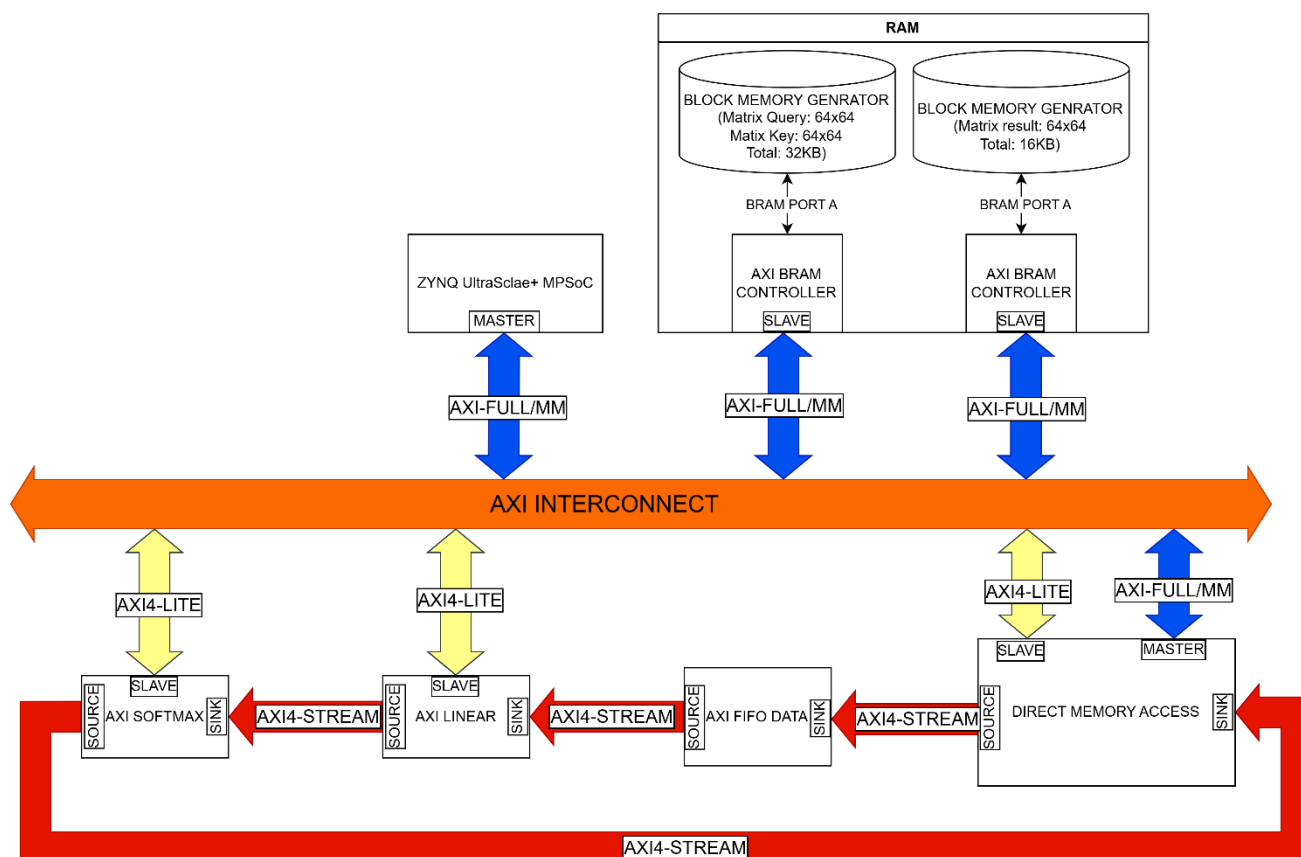
Nhằm giải quyết nút thắt cổ chai về mặt tính toán và tối ưu hóa hệ thống cho các ứng dụng Edge AI, công nghệ FPGA nổi lên như một giải pháp tăng tốc phần cứng lý tưởng nhờ khả năng thiết kế kiến trúc song song mạnh mẽ và khả năng tái cấu hình linh hoạt. Trong khuôn khổ đề án thực tập kỹ sư này, trọng tâm nghiên cứu được đặt vào việc thiết kế, tối ưu hóa và tích hợp hệ thống lõi IP tăng tốc phần cứng chuyên biệt cho bộ Softmax Attention, hướng đến việc triển khai thực tế trên bộ vi xử lý hỗn hợp Zynq UltraScale+ MPSoC (Kit KV260).

Nội dung cốt lõi của đề án là thiết kế một luồng dữ liệu khép kín dựa trên kiến trúc Streaming IP-to-IP. Hệ thống bao gồm hai module gia tốc chính: IP AXI Linear đảm nhiệm phép

nhân cộng tích lũy sử dụng cơ chế truyền tin và IP AXI Softmax được thiết kế đặc biệt nhằm giải quyết các phép chia và hàm mũ phức tạp thông qua các khối bảng tra cứu và bộ chia nghịch đảo thân thiện với phần cứng. Toàn bộ hệ thống được phối hợp nhịp nhàng bởi khốiDMA, sử dụng giao thức AXI4-Stream cho luồng dữ liệu tốc độ cao và AXI4-Lite cho luồng điều khiển. Cơ chế này cho phép bộ tăng tốc hoạt động độc lập, tận dụng tối đa băng thông và giảm tải hoàn toàn cho CPU lõi ARM.

Thông qua đồ án, kiến trúc đề xuất không chỉ được mô phỏng kiểm chứng chặt chẽ tính đúng đắn trên môi trường Vivado, mà còn được tổng hợp và thực thi trực tiếp trên bo mạch KV260 dưới cả hai hệ điều hành Bare-metal và Ubuntu Linux. Các kỹ thuật tối ưu hóa thiết kế phần cứng như cơ chế đường ống bộ đệm kép cũng như chiến lược phân mảnh dữ liệu bằng phần mềm đã được nghiên cứu và đánh giá chi tiết. Mục tiêu cuối cùng là đạt được sự cân bằng tối ưu giữa hiệu năng thông lượng, độ trễ và lượng tài nguyên tiêu thụ trên chip. Kết quả của đồ án đóng góp một thiết kế tham chiếu mang tính thực tiễn cao, tạo tiền đề cho việc đưa các mô hình ngôn ngữ lớn và Transformer tới gần hơn với các thiết bị biên trong hệ sinh thái AI vạn vật.

2. Kiến trúc hệ thống



Hình 1: Sơ đồ kiến trúc bus hệ thống

Thông tin chuẩn giao tiếp của các IP được sử dụng.

Table 1: Thông tin của các IP

Tên IP	Giao diện	Chức năng
Zynq UltraScale+ MPSoC	AXI4-Lite Master, AXI4-Full Master, phần mềm	Chạy Linux OS. Sinh dữ liệu đầu vào (Q, K), lập trình DMA, IP AXI Linear, IP AXI Softmax
DMA	AXI4-Full Master MM2S, AXI4-Full Master S2MM, AXI4-Lite Slave, AXI4-Stream Master MM2S, AXI4-Stream Slave S2MM	Nhận ma trận Key, Query từ SRAM, nạp vào ngõ vào sink của IP Linear. Nhận ma trận kết quả từ IP Softmax, nạp vào SRAM.
AXI FIFO DATA	AXI4-Stream Slave, AXI4-Stream Master	Buffer đệm, phòng trường hợp DMA gửi dữ liệu bị trễ so với dự kiến do nguyên nhân bên ngoài.
AXI Bram Controller	AXI4-Full Slave	Lấy ma trận Key, Query từ SRAM và đưa cho DMA đọc.

AXI Bram Controller	AXI4-Full Slave	Nhận dữ liệu ma trận kết quả từ DMA và ghi vào SRAM
AXI Linear	AXI4-Lite Slave, AXI4-Stream Master, AXI4-Stream Slave	Thực hiện phép tính attention score $S = \frac{Q * K^T}{\sqrt{D \text{ MODEL}}}$
AXI Softmax	AXI4-Lite Slave, AXI4-Stream Master, AXI4-Stream Slave	Tính xác suất Softmax cho từng hàng của ma trận S

3. Bài nghiên cứu liên quan

Trong các nghiên cứu gần đây, việc tăng tốc mô hình Transformer trên FPGA tập trung vào hai thách thức chính: tối ưu khối nhân ma trận và xử lý hiệu quả các hàm phi tuyến như Softmax, LayerNorm, GELU.

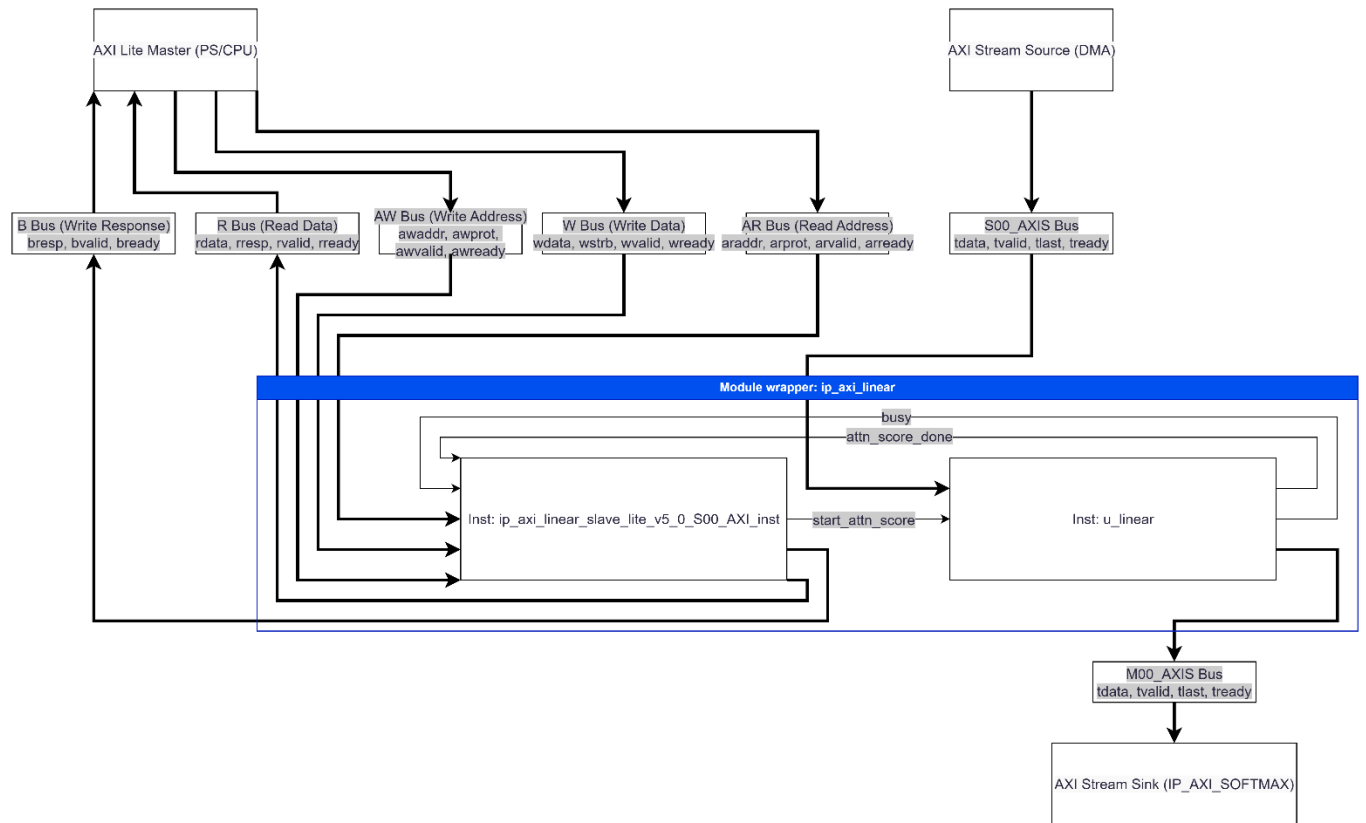
Về hướng thứ nhất, Ye và cộng sự [7] đề xuất một mảng Systolic Array có khả năng tái cấu hình trên nền tảng Xilinx Alveo U250. Ý tưởng cốt lõi là thiết kế phần tử xử lý FPE có thể linh hoạt chuyển đổi giữa hai chế độ: xử lý song song hai phép MAC với dữ liệu 8-bit, hoặc một phép MAC với dữ liệu 16-bit, tất cả trên cùng một khối DSP48, tránh lãng phí tài nguyên phần cứng. Đối với hàm Softmax, nhóm tác giả thay thế phép lũy thừa cơ số e truyền thống bằng cơ chế Radix-2, chuyển toàn bộ phép tính mũ thành các phép dịch bit đơn giản, loại bỏ hoàn toàn bộ tính hàm mũ phức tạp. Kết hợp với bộ lập lịch hai tầng để tự động quản lý luồng dữ liệu cho nhiều kích thước chuỗi đầu vào khác nhau, thiết kế đạt thông lượng 1800 GOP/s, giảm độ trễ 51.3 lần và tiết kiệm năng lượng 54.4 lần so với CPU.

Bên cạnh hướng đi tái cấu hình phần tử tính toán, việc module hóa luồng xử lý Attention kết hợp kỹ thuật phân mảnh ma trận trên bộ nhớ on-chip đóng vai trò quyết định trong việc tối ưu hiệu năng và băng thông. Kabir và cộng sự [8] đã giới thiệu kiến trúc bộ tăng tốc linh hoạt FAMOUS dành riêng cho cơ chế Attention trên các dòng FPGA Xilinx UltraScale+. Nhóm tác giả tách biệt quá trình tính toán Attention thành các module phần cứng độc lập và chuyên biệt hóa, bao gồm: module nhân ma trận $Q \times K^T$, module chuẩn hóa Softmax, và module nhân ma trận trọng số với vector giá trị. Nhằm khắc phục hạn chế về dung lượng bộ nhớ nội bộ khi xử lý các chuỗi đầu vào lớn, bài báo đề xuất chiến lược Tiling ma trận tối ưu, giúp phân bổ đều dữ liệu và tối đa hóa hiệu suất sử dụng của các phần tử xử lý mà không làm tắc nghẽn đường truyền. Triển khai thực tế trên bo mạch Xilinx Alveo U55C cho thấy FAMOUS đạt mức thông lượng tối đa 328 GOP/s, tăng tốc độ thực thi gấp 3.28 lần so với vi xử lý Intel Xeon Gold 5220R và gấp 2.6 lần so với GPU NVIDIA V100, đồng thời vượt trội 1.3 lần so với các thiết kế tăng tốc Attention trên FPGA hiện hành.

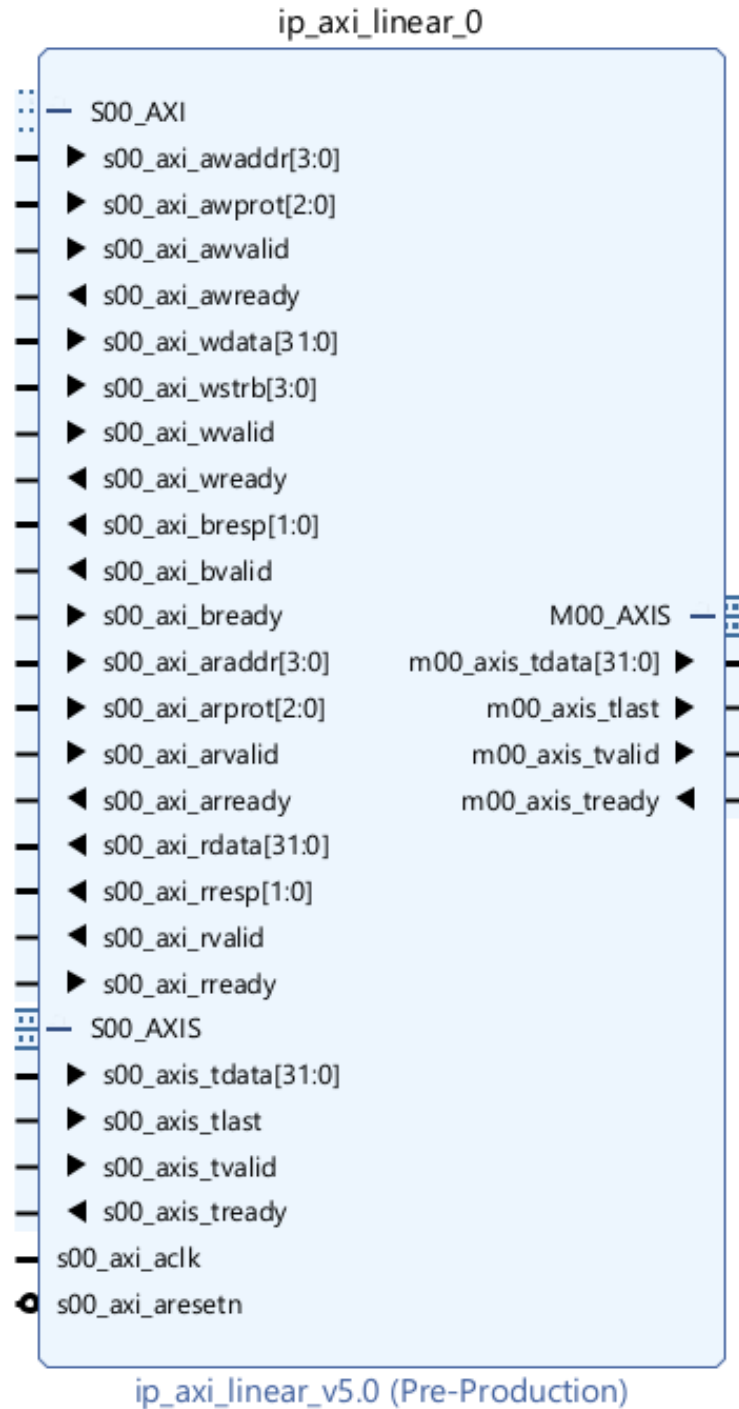
II. KIẾN TRÚC PHẦN CỨNG

1. Cấu trúc IP AXI Linear

1.1. Tổng quan module wrapper



Hình 2: Sơ đồ khối module wrapper `ip_axi_linear`



Hình 3: Module `ip_axi_linear`

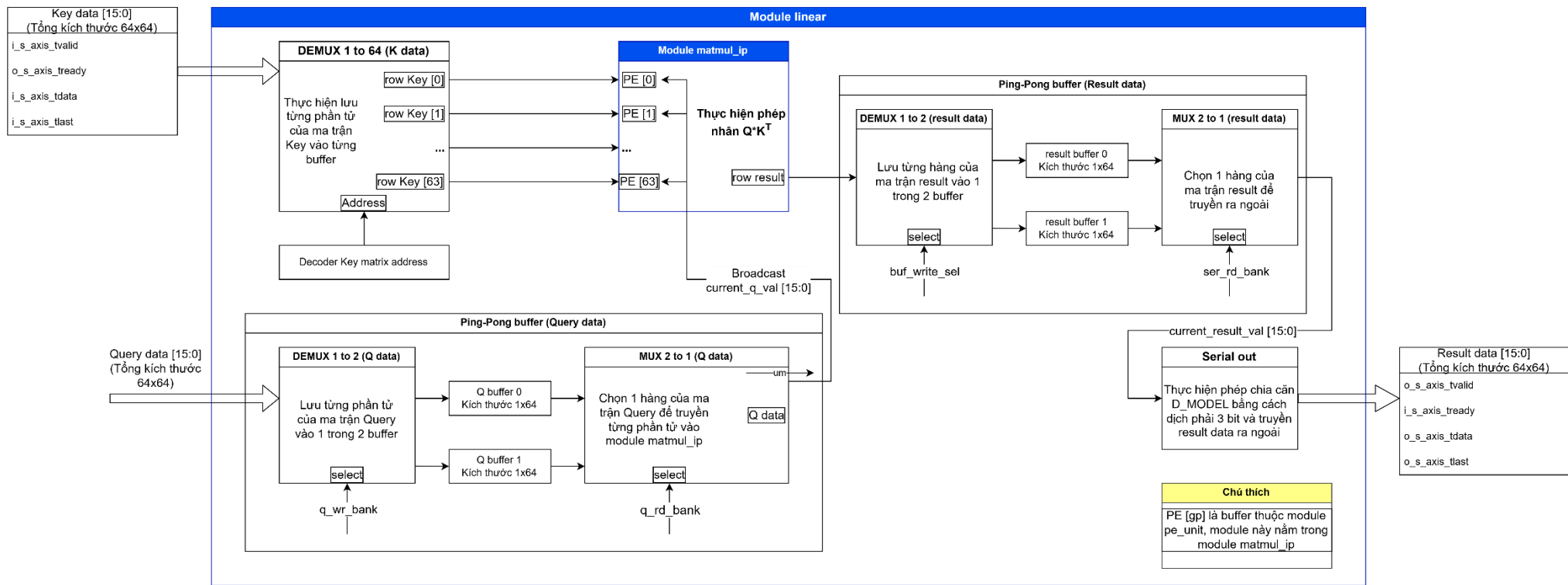
1.2. Tổng quan kiến trúc core `linear sv`

IP AXI Linear là khối tăng tốc phần cứng đảm nhận phép tính nhân ma trận và chia tỷ lệ cho cơ chế Attention. Kiến trúc IP phân tách rõ ràng luồng điều khiển và luồng dữ liệu. Về điều

khien, bộ xử lý ARM Cortex-A53 giao tiếp với lõi thuật toán thông qua bus AXI4-Lite để thiết lập cấu hình, kích hoạt và giám sát trạng thái thông qua hệ thống thanh ghi. Về dữ liệu, IP nhận luồng dữ liệu đầu vào tốc độ cao từ DMA qua AXI4-Stream Slave. Lõi thuật toán xử lý luồng dữ liệu này và đẩy ngay kết quả ra AXI4-Stream Master. Ngõ ra này được nối trực tiếp vào IP AXI Softmax, tạo thành một pipeline liên hoàn ở cấp hệ thống giúp tối đa hóa thông lượng và loại bỏ hoàn toàn độ trễ thất cổ chai bộ nhớ.

Table 2: Các module trong IP AXI Linear

File	Module	Chức năng
ip_axi_linear.sv	ip_axi_linear (wrapper)	Nhận tín hiệu từ bên ngoài vào, kết nối tín hiệu đó vào các submodule bên dưới. (Do Xilinx vivado sinh ra)
ip_axi_linear_slave_lite_v5_0_S00_AXI.sv	Ip...0_S00_AXI	Giao diện giao tiếp AXI4-Lite Slave, bao gồm các kênh Write Address, Write data, Read address, Read data. (Do Xilinx vivado sinh ra)
Linear.sv	Linear	Top-level lõi tính toán, kết nối FSM, nhận ma trận Key, Query, truyền ma trận attention score cho IP Softmax.
Matmul_ip.sv	Matmul_ip	Gọi 64 PE (64 module pe_unit) chạy song song, truyền ma trận Key và Query vào pe_unit, truyền ma trận kết quả ra ngoài.
	pe_unit (submodule)	Thực hiện phép nhân cộng tích lũy của phép nhân ma trận $Q * K^T$.

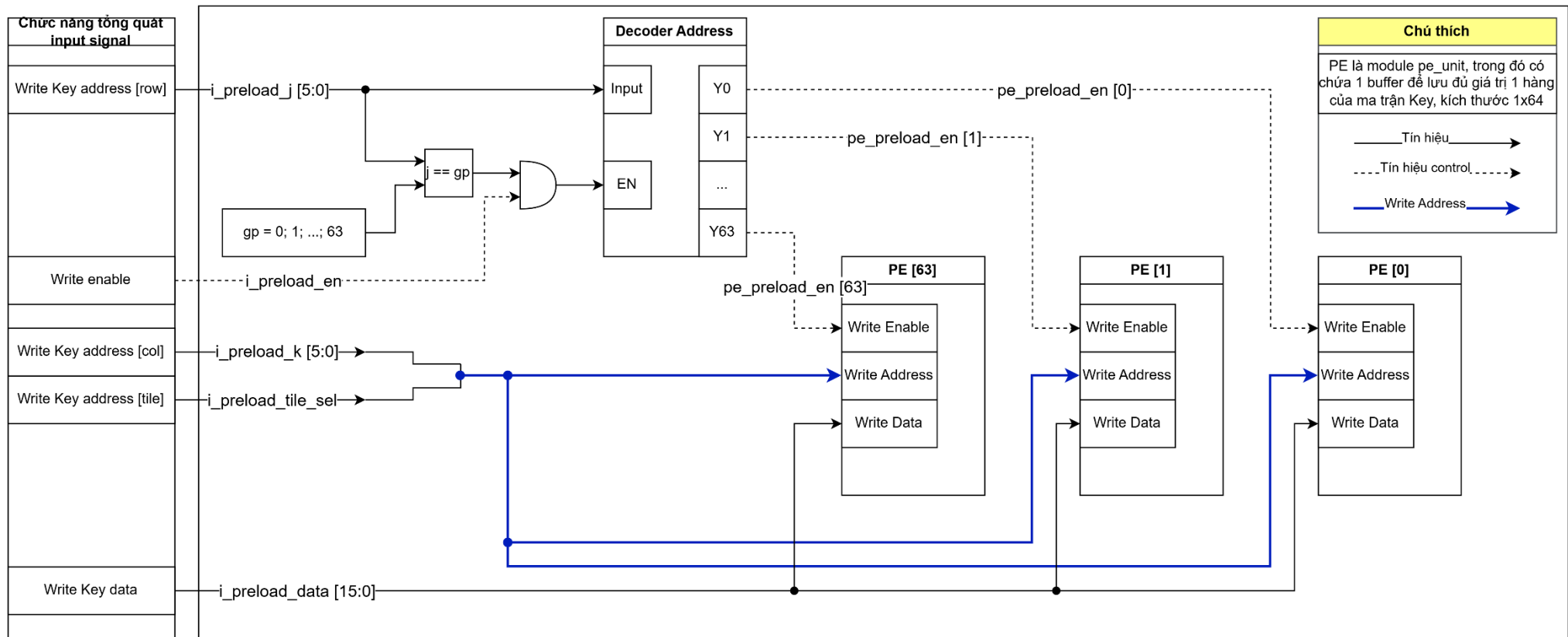


Hình 4: Sơ đồ khối của file linear.sv

Table 3: Các khối chức năng trong file linear.sv

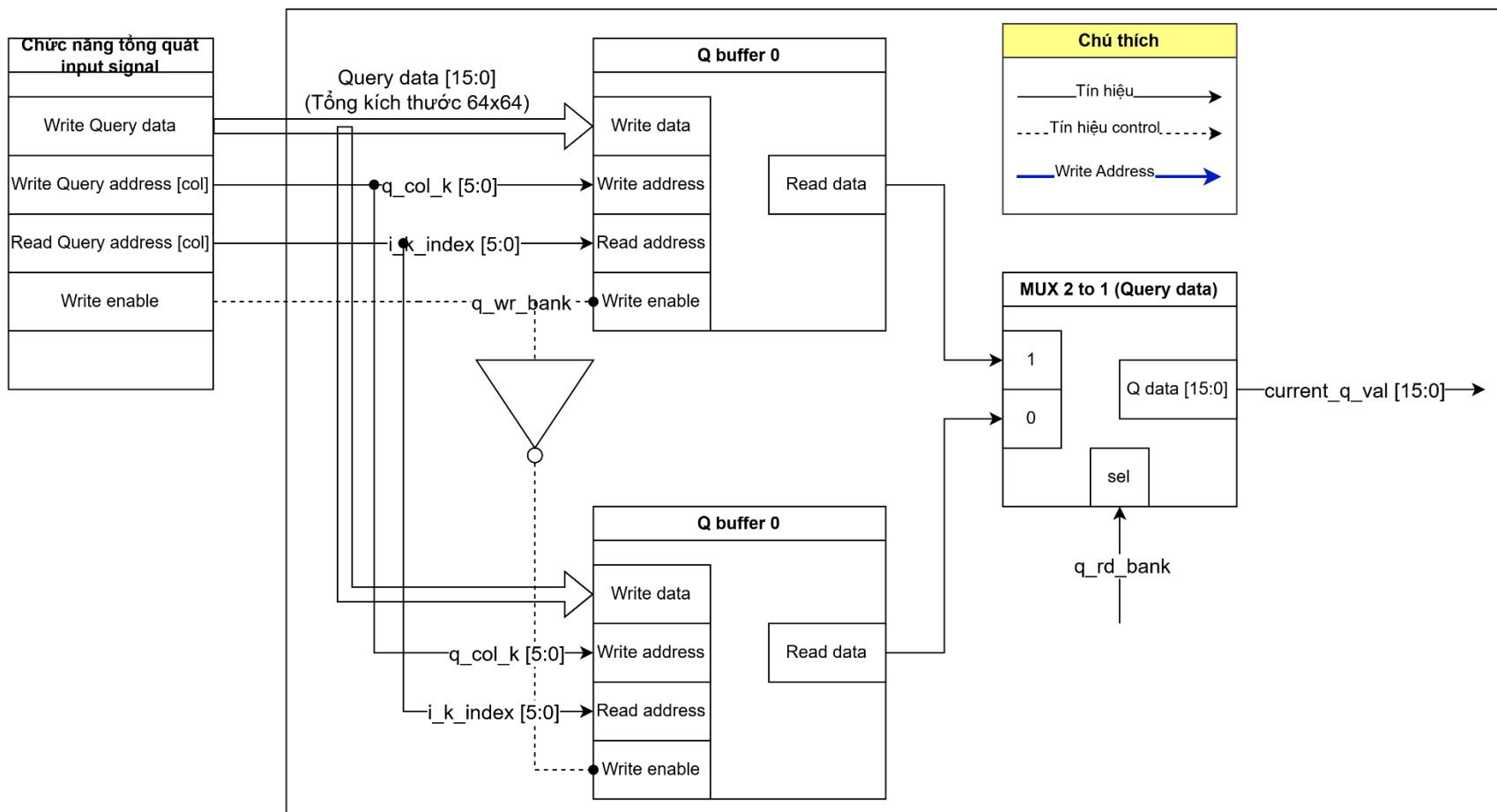
Tên khối chức năng	Chức năng khối
DEMUX 1 to 64 (K data)	Nhận từng giá trị của ma trận Key, độ rộng 16 bit, tổng số lượng 64x64, lưu từng giá trị này vào các buffer trong module pe_unit, mỗi buffer có độ rộng 1 hàng (1x64).
Decoder Key matrix address	Tạo địa chỉ address để kết nối vào address của khối demux ma trận Key.
Ping-Pong buffer (Query data)	Nhận từng giá trị của ma trận Query, độ rộng 16 bit, tổng số lượng 64x64, lưu vào các buffer và truyền giá trị này vào các buffer pe trong module pe_unit.
Module matmul_ip	Thực hiện phép tính $Q * K^T$
Ping-Pong buffer (Result data)	Nhận từng giá trị của ma trận result, độ rộng 16 bit, tổng số lượng 64x64, gửi tới khối serial out để tính phép chia.
Serial out	Thực hiện phép chia $\sqrt{D_MODEL} = 8$ bằng cách dịch trái 3 bit, sau đó lưu vào các buffer và truyền giá trị này ra ngoài.

Sơ đồ khối của khối DEMUX 1 to 64 (K data)



Hình 5: Sơ đồ khối chức năng của khối DEMUX 1 to 64 (K data)

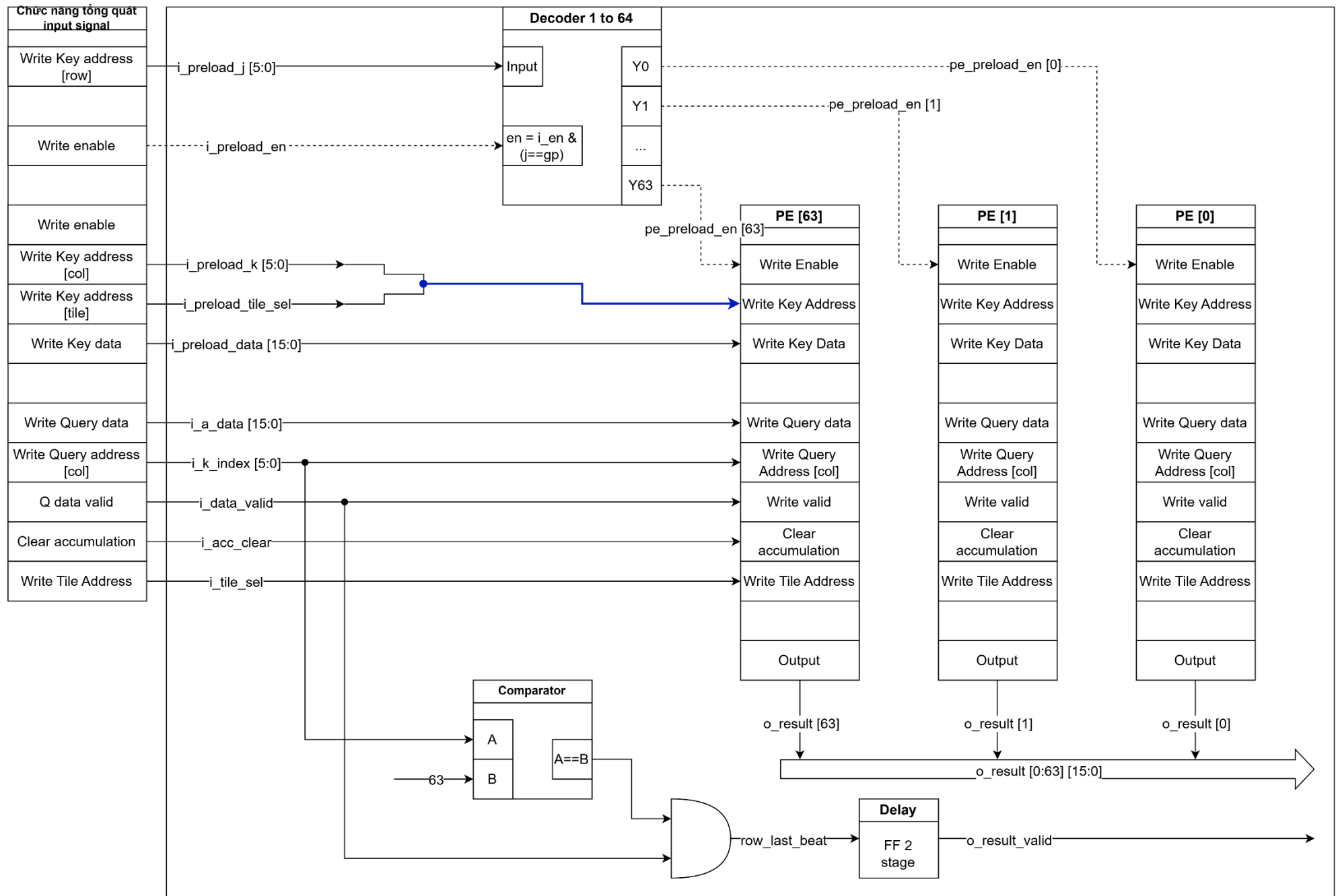
Tên khối chức năng	Chức năng khối
$gp = 0; 1; 2; \dots; 63$	Khởi tạo 64 instance cùng lúc, code: <code>genvar gp; generate for (gp = 0; gp < N_PE; gp++),</code> 64 instance này là 64 module <code>pe_unit</code> (submodule trong <code>matmul_ip</code>).
Decoder Address	Nhận tín hiệu Write Key address [row] rồi decoder ra 64 tín hiệu đầu ra, tín hiệu enable cho mỗi PE là khi address [row] == gp.
PE [gp]	Mỗi PE [gp] là một module <code>pe_unit</code> , có các tín hiệu phục vụ cho việc write Key data vào buffer trong module này như <code>Wen</code> ; <code>Waddr</code> ; <code>Wdata</code> .



Hình 6: Sơ đồ khối chức năng của khối Ping-Pong buffer Query data

Table 4: Các khối chức năng của khối Ping-Pong buffer Query data

Tên khối chức năng	Chức năng khối
Q_buffer_0; q_buffer_1	Buffer dùng để lưu giá trị phần tử ma trận Query, mỗi buffer có độ rộng bằng đúng 1 hàng, 1x64, 2 buffer này luân phiên thay đổi để nhận 1 trong hai chức năng: - Nhận data Query từ bên ngoài đến khi đầy buffer - Đọc data Query khi buffer đã đầy, gửi tín hiệu này vào module matmul_i
MUX 2 to 1 (Query data)	Chọn 1 trong hai buffer để thực hiện nhiệm vụ Đọc.

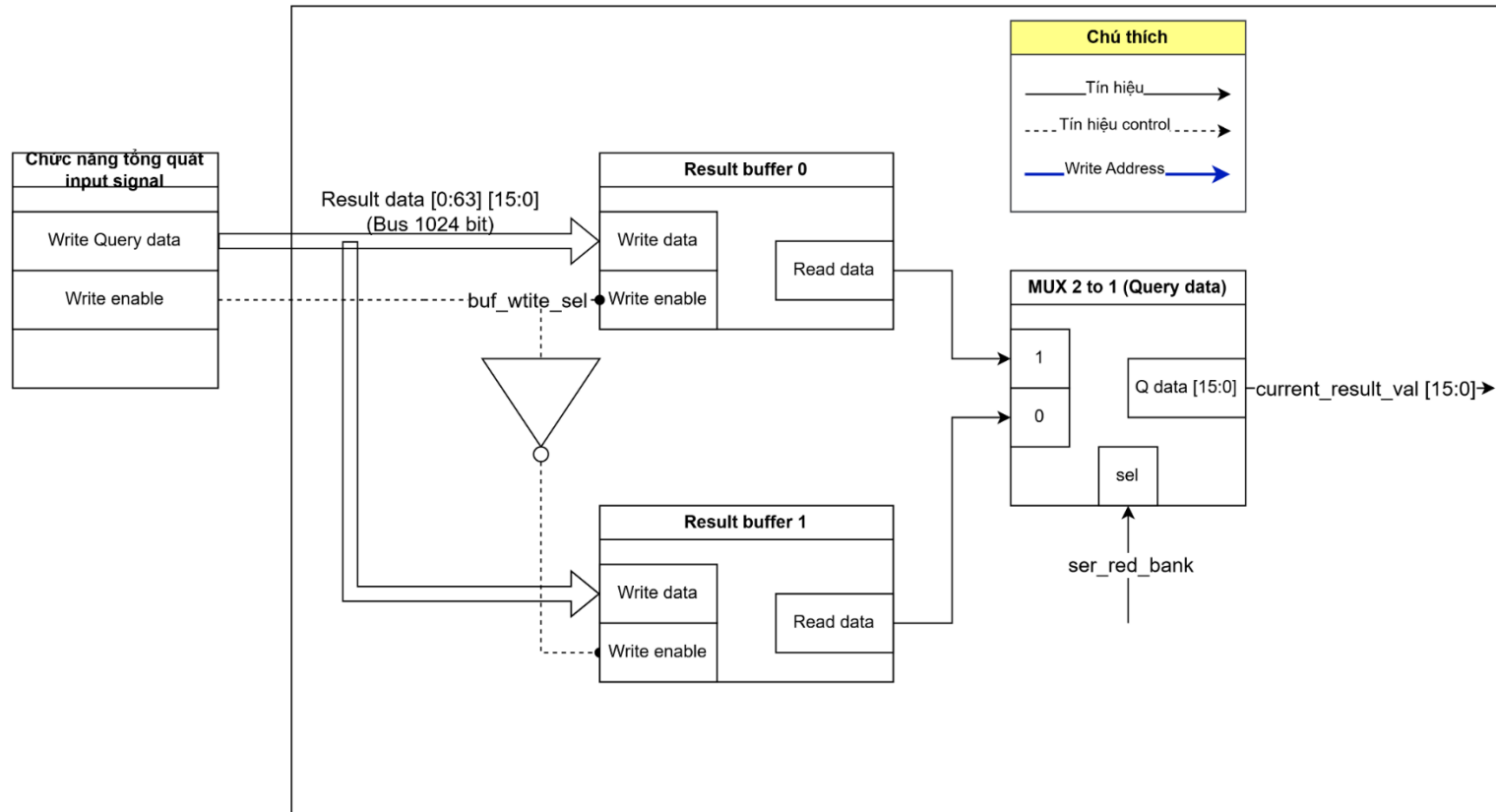


Hình 7: Sơ đồ khối chức năng của khối module matmul_ip

Table 5: Các khối chức năng của khối module matmul_ip

Tên khối chức năng	Chức năng khối
Decoder Address	Nhận tín hiệu Write Key address [row] rồi decoder ra 64 tín hiệu đầu ra, tín hiệu enable cho mỗi PE là khi address [row] == gp.
PE [gp]	Module pe_unit thực hiện 2 nhiệm vụ chính là ghi sẵn giá trị Key và broadcast giá trị Query để nhân cộng tích lũy. - Mỗi PE [gp] là một module pe_unit, có các tín hiệu phục vụ cho việc write Key data vào buffer trong module này như Wen; Waddr; Wdata. - Nhóm tín hiệu Write Query Address; Write Query Data; Writ valid dùng để broadcast từng phần tử Query đến cùng lúc 64 PE, sau đó pe_unit sẽ thực hiện phép nhân cộng tích lũy để tạo ra các phần tử của 1 hàng. Như vậy, khi hoàn thành broadcast đủ 1 hàng của ma trận Query thì sẽ tạo ra song song đúng 64 phần tử của ma trận kết quả. Khi có 1 hàng được tính xong, module matmul_ip sẽ truyền row này ra ngoài.
Comparator	So sánh tọa độ cột của Query data với 63 hay chưa, nghĩa là đã broadcast đủ 1 hàng hay chưa, nếu đủ thì xem như đã tính xong 1 hàng ma trận kết quả và gửi tín hiệu valid của ma trận kết quả.
Delay	Thực hiện delay 2 cycles.

Ping-pong result data

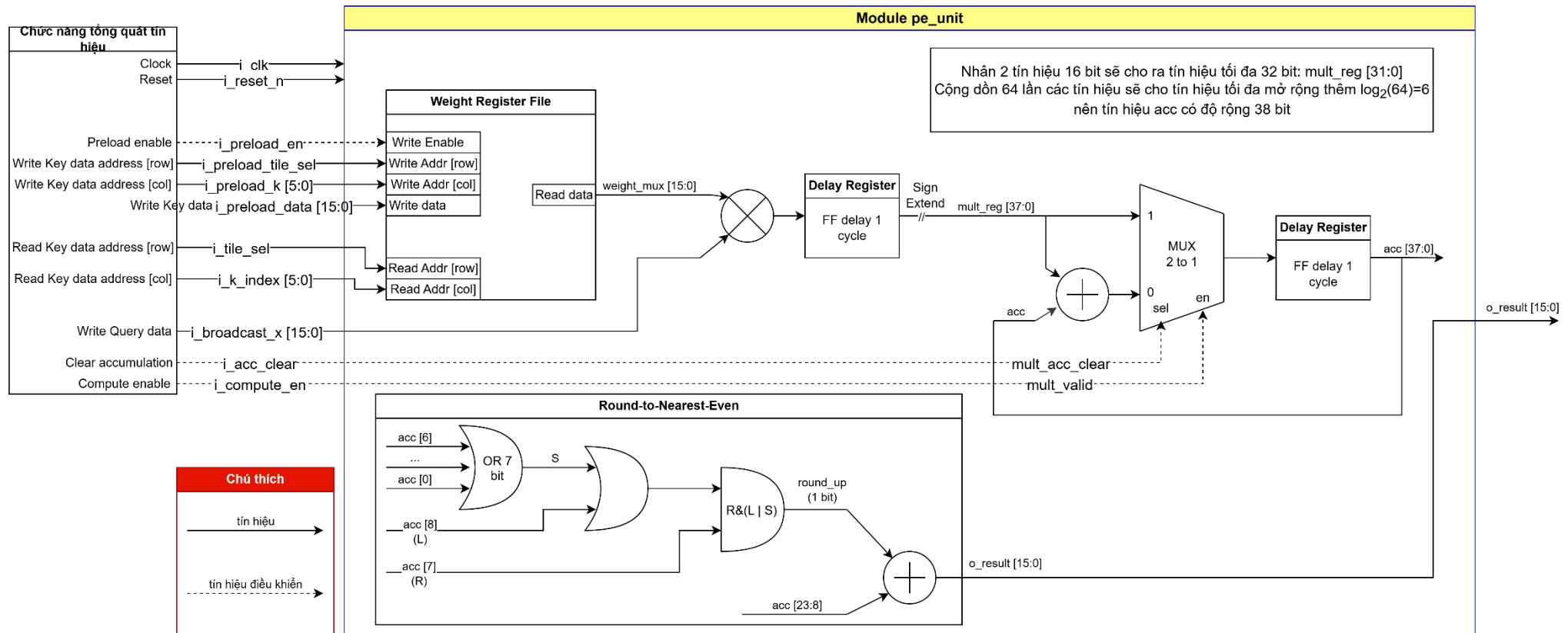


Hình 8: Sơ đồ khối chức năng của khối Ping-pong result data

Table 6: Các khối chức năng của khối Ping-pong result data

Tên khối chức năng	Chức năng khối
result_buffer_0; result_buffer_1	Buffer dùng để lưu giá trị phần tử ma trận kết quả, mỗi buffer có độ rộng bằng đúng 1 hàng, 1x64, 2 buffer này luân phiên thay đổi để nhận 1 trong hai chức năng: - Nhận data result từ bên module matmul_ip đến khi đầy buffer - Đọc data result khi buffer đã đầy, gửi tín hiệu này vào khối serial out
MUX 2 to 1 (Query data)	Chọn 1 trong hai buffer để thực hiện nhiệm vụ Đọc.

Module pe_unit



Hình 9: Sơ đồ khối chức năng của khối pe_unit

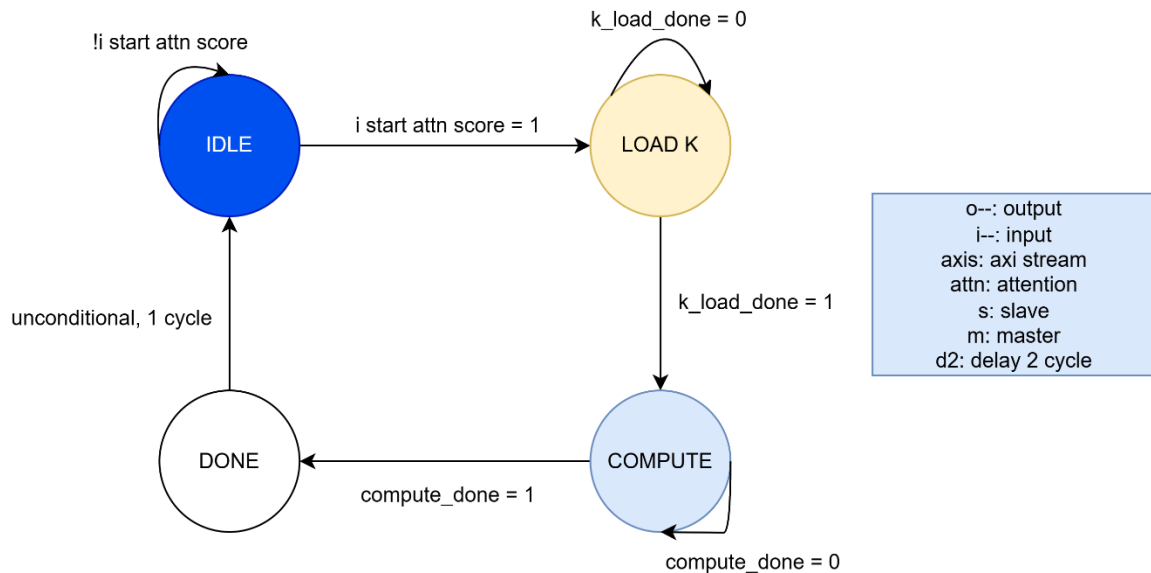
Table 7: Các khối chức năng của khối module pe_unit

Tên khối chức năng	Chức năng khối
Weight register file	Lưu một hàng của ma trận Key vào buffer bên trong, buffer có các tín hiệu Write enable, write address, write data, read address, read data.
Delay register	Flip-flop delay 1 cycle
Round-to-nearest-even	Phương pháp làm tròn gần nhất số chẵn, nhằm mục đích giảm thiểu sai số cộng dồn.

Phương pháp round-to-nearest-even

Acc [8] (L)	Acc [7] (R)	OR 7 bit acc [6:0] (S)	Phần thập phân so với 0,5	Làm tròn	Round_up
-	0	0	= 0	Giữ nguyên	0
-	0	1	< 0,5	Làm tròn xuống	0
-	1	1	> 0,5	Làm tròn lên	1
0	1	0	= 0,5	Phần nguyên là số chẵn nên giữ nguyên	0
1	1	0	= 0,5	Phần nguyên là số lẻ nên làm tròn lên số chẵn	1

1.3. Tổng quan FSM IP AXI Linear



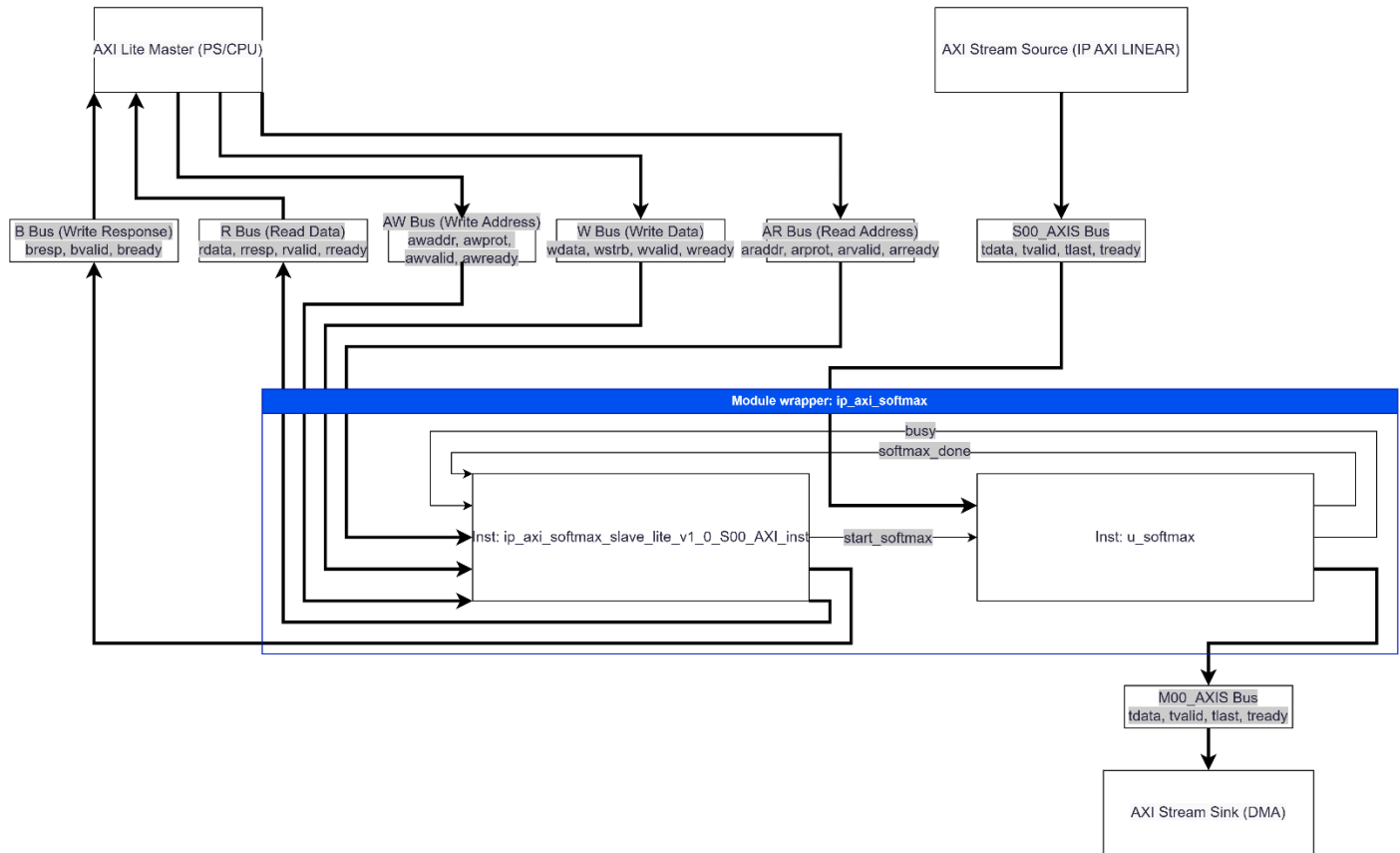
Hình 10: Sơ đồ FSM Linear dạng Moore

Table 8: Bảng chức năng FSM của file linear.sv

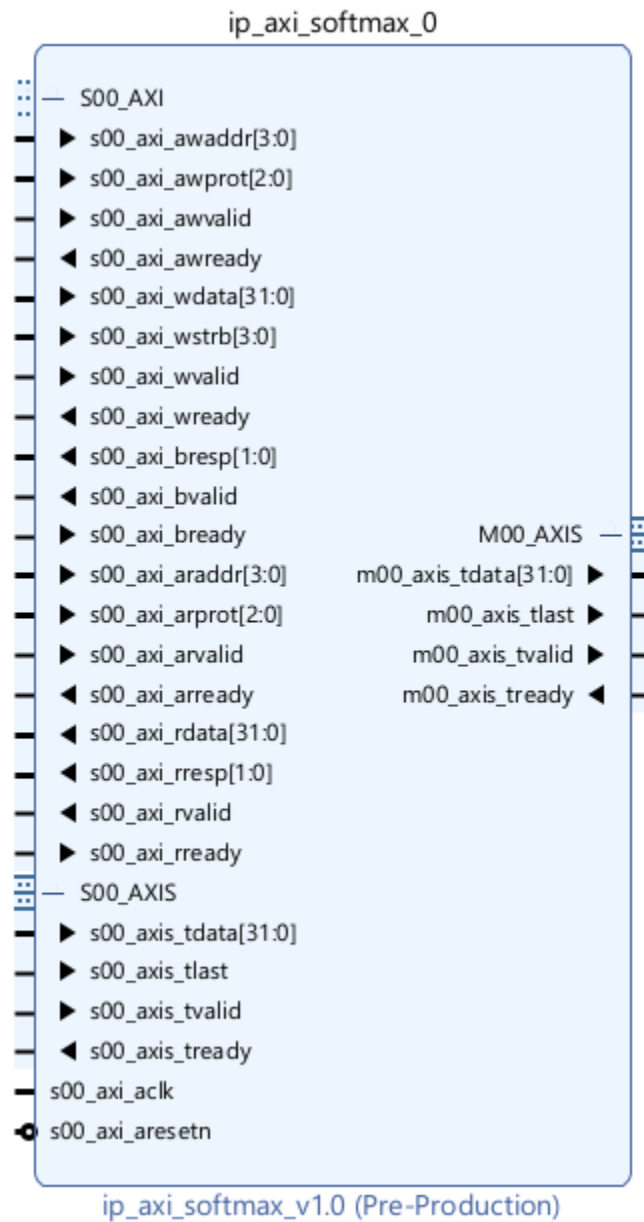
Tên trạng thái	Chức năng
ST_IDLE	Trạng thái tĩnh, đợi tín hiệu start thông qua AXI Lite để chuyển sang state LOAD K.
ST_LOAD_K	DMA đọc data ma trận Key từ IP BRAM, sau đó gửi dữ liệu vào ip linear, data sẽ được preload vào trong các pe của module matmul_ip theo cấu trúc mỗi pe chứa một hàng của ma trận Key. Số pe được tham số hóa, nhỏ hơn D HEAD, nếu số pe nhỏ hơn D HEAD thì áp dụng phương pháp tilling. Do chọn $N_{PE} = D_{HEAD}$ nên số tile là 1.
ST_COMPUTE	DMA MM2S thực hiện stream data Q vào trong các pe để tính toán, mỗi lần thực hiện stream 1 hàng Q gồm d model = 64 phần tử, các pe sẽ tính toán và cho ra cùng lúc 1 hàng của score (Nếu $N_{PE} = D_{HEAD}$), nếu $N_{PE} < D_{HEAD}$ thì thực hiện phương pháp tilling đến khi tính xong 1 hàng thì xuất ra cổng stream đến DMA S2MM, phương pháp tính được áp dụng là SIMD-style inner product.
ST_DONE	Kết thúc quá trình tính, gửi tín hiệu done.

2. Cấu trúc IP AXI Softmax

2.1. Tổng quan module wrapper

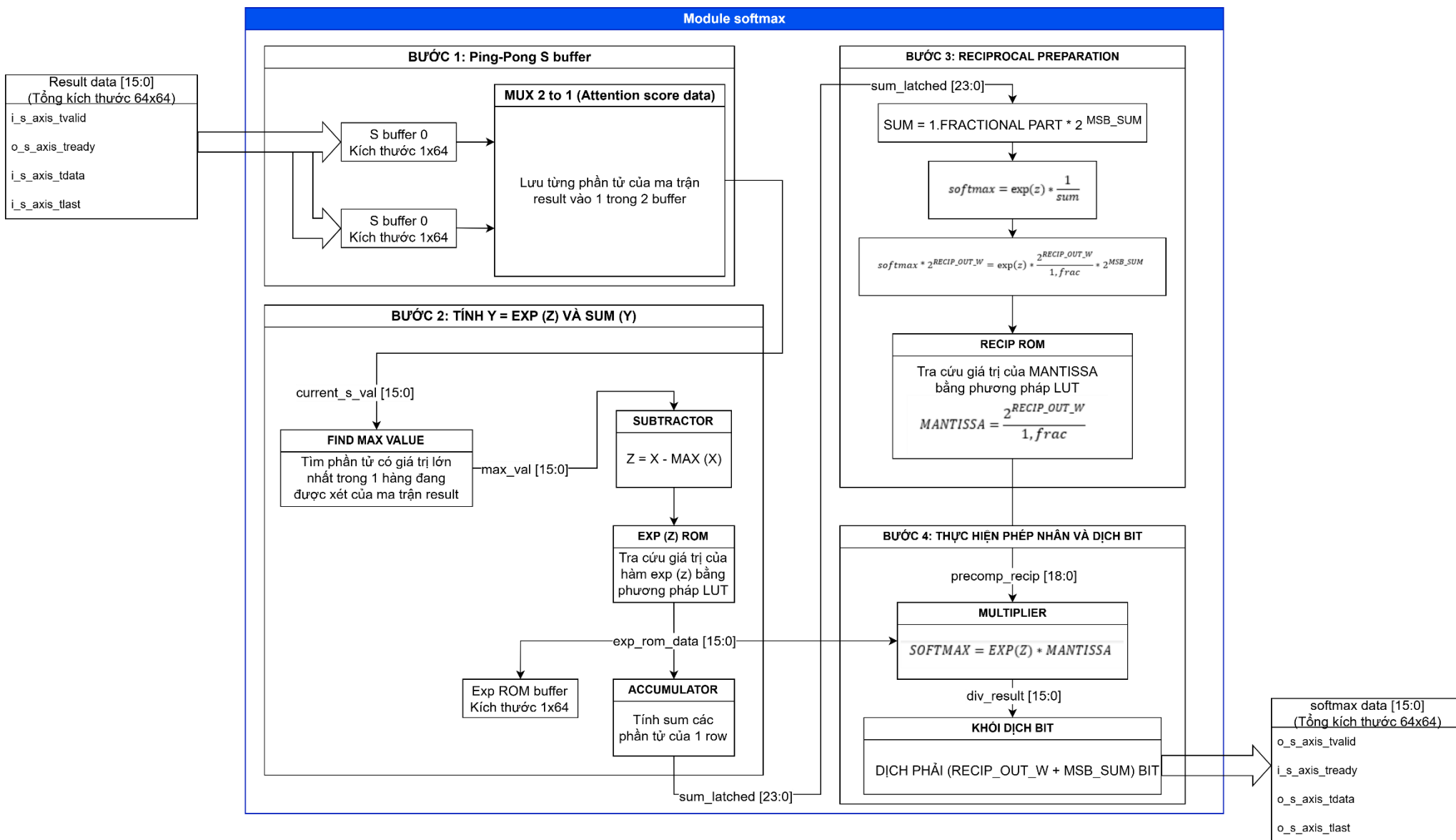


Hình 11: Sơ đồ khối module wrapper ip_axi_softmax



Hình 12: module `ip_axi_softmax`

2.2. Tổng quan kiến trúc core softmax.sv



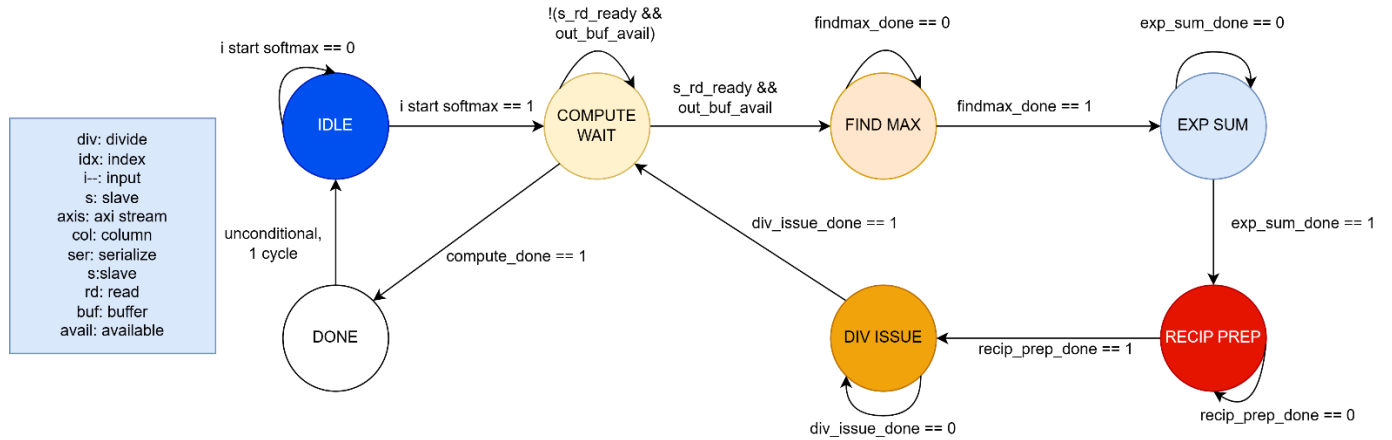
Hình 13: Sơ đồ khối file softmax.sv

Table 9: Các khối chức năng của file softmax.sv

Tên khối chức năng	Chức năng khối
MUX 2 to 1	Thực hiện lưu từng phần tử của ma trận result vào 1 trong 2 buffer
FIND MAX VALUE	Tìm phần tử có giá trị lớn nhất trong hàng của ma trận đang xét
SUBTRACTOR	Thực hiện chuẩn hóa tín hiệu bằng cách $Z = X - MAX(X)$
EXP (Z) ROM	Khối RAM có lưu các giá trị của hàm exp (z)
ACCUMULATOR	Tính sum các phần tử trong hàng
RECIP_ROM	Khối RAM có lưu các giá trị MANTISSA
MULTIPLIER	Thực hiện phép nhân $softmax = exp(z) * mantissa$
KHỐI DỊCH BIT	Dịch phải lượng bit đúng bằng $RECIP_OUT_W + MSB_SUM$ để trả về đúng giá trị softmax

IP Softmax gồm 2 phần, control plane và data plane. Control plane được SoC điều khiển với interface AXI Lite, data plane được kết nối AXI Stream với IP Linear và IP DMA.

2.3. FSM IP Softmax



Hình 14: FSM IP Softmax dạng Moore

Table 10: Bảng chức năng FSM của file softmax.sv

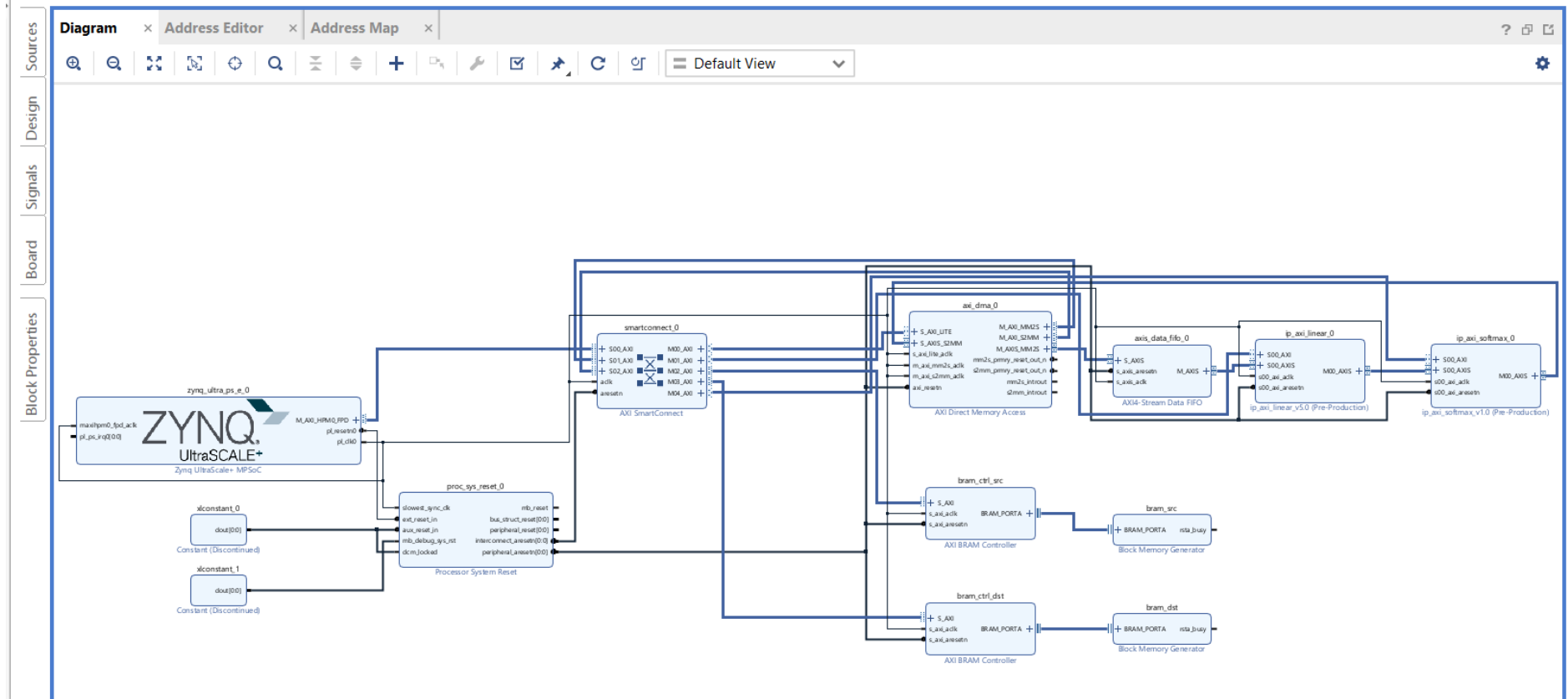
Tên trạng thái	Chức năng
ST_IDLE	Hệ thống đợi tín hiệu <code>i_start_softmax</code> kích hoạt từ AXI Lite để khởi động toàn bộ quá trình và chuyển sang trạng thái chờ dữ liệu.
ST_COMPUTE_WAIT	Điểm chốt chặn đồng bộ Ping-Pong. FSM sẽ đứng chờ ở đây để đảm bảo 2 điều kiện: Buffer đầu vào đã nạp xong ít nhất 1 hàng dữ liệu (<code>s_rd_ready = 1</code>) VÀ Buffer đầu ra đang trống (<code>out_buf_avail = 1</code>). Nếu thỏa mãn, chuyển sang pha tính toán. Nếu đã xử lý xong toàn bộ ma trận, chuyển sang ST_DONE.

ST_FIND_MAX	Pha tính toán 1: Tìm giá trị lớn nhất. Để chống tràn số (overflow) khi tính hàm mũ, FSM duyệt tuần tự qua 64 phần tử của hàng hiện tại để tìm và chốt giá trị lớn nhất vào thanh ghi max_val. Mất 64 chu kỳ xung nhịp.
ST_EXP_SUM	Pha tính toán 2: Tính $e^{x-\max(x)}$ và cộng dồn mẫu số. FSM tiếp tục duyệt qua 64 phần tử, thực hiện phép trừ với max_val, sau đó đưa vào khối EXP_ROM để tra cứu giá trị hàm mũ. Kết quả được đưa vào bộ đệm exp_row_buf, đồng thời cộng dồn liên tục vào thanh ghi sum_acc để làm mẫu số. Mất 64 chu kỳ xung nhịp.
ST_RECIP_PREP	Pha tính toán 3: Tính toán nghịch đảo mẫu số. Mạch tìm vị trí bit 1 cao nhất của tổng sum_acc, sau đó đưa vào khối RECIP_ROM để lấy hằng số nghịch đảo và tính toán độ dịch bit. Pha này tốn 2 chu kỳ để chờ dữ liệu đồng bộ từ ROM.
ST_DIV_ISSUE	Pha tính toán 4: 8-Way SIMD Divide. Thay vì tốn 64 nhịp để chia từng số, mạch lấy đồng loạt 8 phần tử từ exp_row_buf ra nhân với hằng số nghịch đảo và dịch bit. Ghi 8 kết quả vào Ping-Pong Buffer đầu ra cùng một lúc. Do xử lý song song 8 đường, pha này kết thúc cực nhanh chỉ trong 8 chu kỳ xung nhịp.
ST_DONE	Kết thúc quá trình tính toán toàn bộ ma trận. Bật cờ o_softmax_done báo cho CPU/DMA biết, sau đó tự động quay về trạng thái ST_IDLE.

III. MÔ PHỎNG TRÊN VIVADO

1. Cấu trúc thực thi

Cấu trúc block design:



Hình 15: Block Design mô phỏng Vivado

Table 11: Thông tin các IP được sử dụng trong testbench

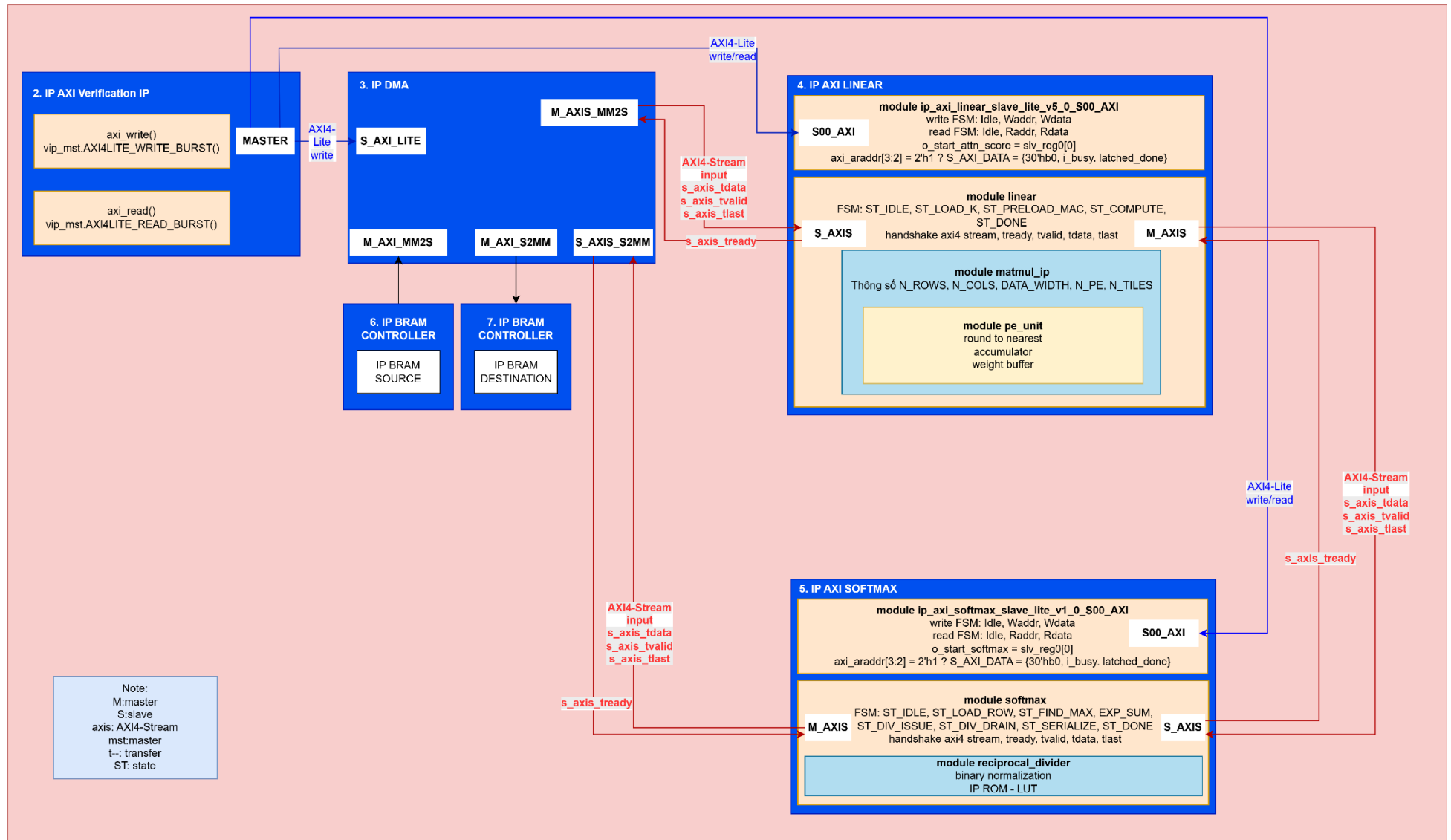
Tên IP	Type	Interface	Mô tả
AXI Verification IP	Master	AXI Full AXI Lite	Control plane của các IP DMA, Linear, Softmax.
Smart connect	Axi interconnect		Tạo các cổng slave, master để master và slave truyền, nhận lệnh.
DMA	Slave	AXI Lite AXI Stream	Được AXI VIP điều khiển init qua AXI Lite. Nhận data output từ IP Softmax qua AXIS.
	Master	AXI Stream	Push data K, Q qua AXIS tới IP Linear.
Linear	Slave	AXI Stream	Nhận K, Q từ DMA.
	Master	AXI Stream	Push Attention score tới Ip Softmax qua AXIS.
Softmax	Slave	AXI Stream	Nhận Attention Score qua AXIS.
	Master	AXI Stream	Push softmax score tới DMA qua AXIS.
BRAM Controller SRC	Slave	AXI	Controller ip BRAM SRC, DMA thực hiện MM2S để lấy data.
BRAM SRC			Lưu K, Q data
BRAM Controller DST	Slave	AXI	Controller ip BRAM DST, DMA thực hiện S2MM để ghi data.
BRAM DST			Lưu Softmax score data.

Địa chỉ và kích thước bộ nhớ cho các IP:

Table 12: Địa chỉ các IP được sử dụng trong testbench

Tên IP	Địa chỉ	Kích thước
DMA	0x4000_0000	64K
Linear	0x4001_0000	64K
Softmax	0x4002_0000	64K
BRAM Controller SRC	0x0	32K
BRAM Controller DST	0x1_0000	16K

2. Cấu trúc mô phỏng



Hình 16: Sơ đồ khối thực hiện testbench

Testbench thực hiện xác thực end-to-end pipeline `ip_axi_linear` → softmax bằng phương pháp so sánh với golden model, thông qua AXI VIP điều khiển DMA và IP dưới test. Flow gồm 4 giai đoạn tuần tự:

Giai đoạn 1 Chuẩn bị dữ liệu (Init): Task `init_src_data()` đọc dữ liệu K, Q từ file `.mem` (dump từ golden model Python: `k_ram.mem`, `q_ram.mem`) và golden score kỳ vọng từ `golden_softmax.mem` vào mảng `golden_score[]` trong testbench, dùng làm reference cho bước so sánh cuối.

Giai đoạn 2 Preload BRAM nguồn: Task `preload_bram_src()` ghi trực tiếp K, Q data vào BRAM src thông qua giao diện AXI-Lite của AXI VIP, tại địa chỉ cố định `K_BASE = SRC_BASE`, `Q_BASE = SRC_BASE + NUM_BYTES_K`. Đây là bước nạp sẵn dữ liệu vào memory-mapped source trước khi kích hoạt DMA, mô phỏng đúng luồng dữ liệu thực tế (memory → DMA → IP) thay vì đưa thẳng qua testbench.

Giai đoạn 3 Vận hành DMA:

- `dma_reset()`: assert reset bit qua AXI VIP, đưa DMA về trạng thái sạch trước khi test, tránh trạng thái tồn dư từ lần chạy trước ảnh hưởng kết quả.
- `dma_start_transfer()`: chuỗi thao tác điều khiển 2 kênh DMA tuần tự
 1. Kênh MM2S lần 1: set `MM2S_SA = K_BASE`, assert start signal của `ip_axi_linear`, set `length = Depth_K × 4 byte`, chờ `wait_mm2s_done()`.
 2. Clear bit IOC trong `DMASR`, re-assert start signal IP để chuẩn bị nhận beat tiếp theo.
 3. Kênh S2MM đồng thời được cấu hình: `S2MM_DA = DST_BASE`, `length = Softmax_score_depth × 4 byte`, nhận kết quả từ output stream của IP ghi ngược vào BRAM đích.
 4. Kênh MM2S lần 2: set `MM2S_SA = Q_BASE`, `length = Depth_Q × 4`, đẩy tiếp Q data vào IP.
 5. Chờ đồng thời `wait_mm2s_done()` và `wait_s2mm_done()`, đảm bảo cả luồng đẩy vào và luồng nhận kết quả ra đều hoàn tất trước khi sang bước so sánh.

Giai đoạn 4 So sánh kết quả: Task `compare_bram_dst_with_golden_model()` đọc từng word từ `DST_BASE` qua AXI VIP, so sánh với `golden_score[i]` đã nạp ở Giai đoạn 1. Với mỗi phần tử không khớp, in ra chỉ số, giá trị kỳ vọng, giá trị thực đo, và độ lệch để phục vụ debug, nếu toàn bộ khớp thì in PASS.

3. Kết quả mô phỏng

Thực hiện mô phỏng với bộ thông số đầu vào như sau:

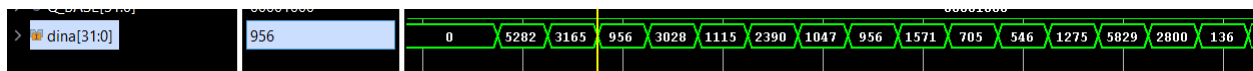
Table 13: Thông số chuẩn hóa khi mô phỏng testbench

Parameter	Value
D MODEL	64
SEQ LEN	64
D HEAD	64
N PE	64
DATA WIDTH	16

```
init data src done
init src data time: 0.00 ns
preload data into bram src done
preload_bram_src time: 2294090.00 ns
dma_reset time: 1320.00 ns
DEBUG: MM2S_DMASR right after reset = 0x00000001
mm2s done
mm2s done
[2441285 ns] [LINEAR IP] Inference completed! Total execution time: 14441 clock cycles
[2444175 ns] [SOFTMAX IP] Inference completed! Total execution time: 9700 clock cycles
s2mm done
dma transfer done
dma_start_transfer time: 149160.00 ns
PASS: all output words match golden model
Wrote RTL output mem file: E:/DOWNLOAD/HCMUT/TTRS/src/mem files/rtl/softmax_score.mem
compare and write file time: 2375680.00 ns
Executing Axi4 End Of Simulation checks
```

Hình 17: Kết quả tcl của tb và giá trị rtl softmax được lưu trong mem file

Kết quả mô phỏng cho đúng softmax 16x16 phần tử giống với golden softmax score, kết quả waveform cũng cho các giá trị được push vào trong IP BRAM DST.

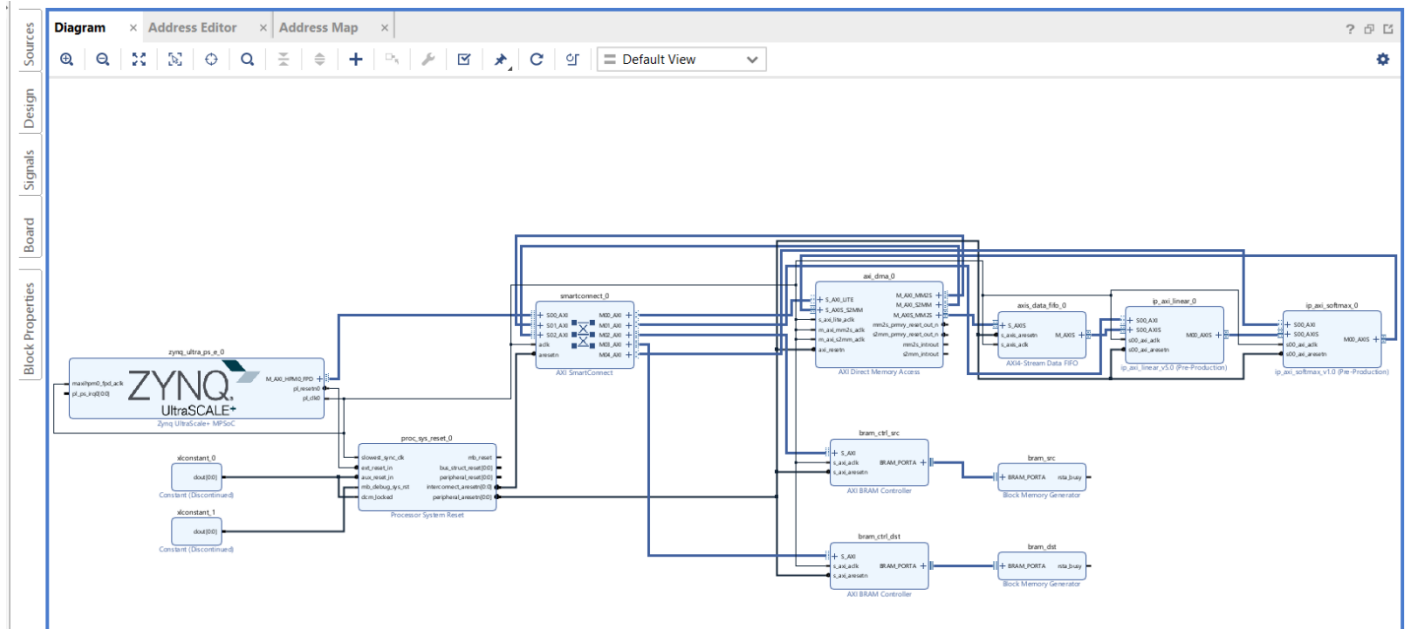


Hình 18: Softmax data được push vào BRAM DST từ DMA S2MM

IV. THỰC HIỆN TRÊN PHẦN CỨNG FPGA KV260

1. Cấu trúc thực thi

Cấu trúc Block Design:



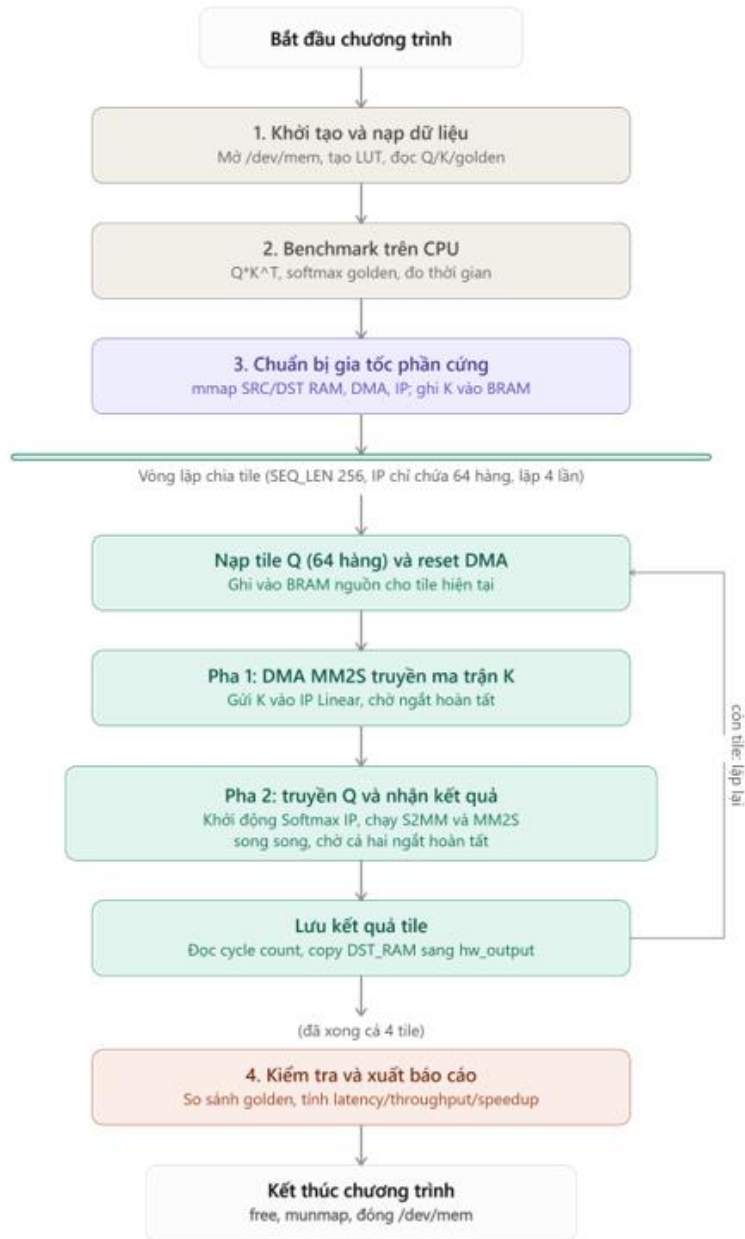
Hình 19: Cấu trúc Block Design

Cấu trúc mô phỏng sẽ bao gồm các ip sau:

Table 14: Thông tin các IP được sử dụng khi thực thi trên KV260

Tên IP	Type	Interface	Mô tả
Zynq UltraScale+ MPSoc	Master	AXI Full AXI Lite	Control plane của các IP DMA, Linear, Softmax.
Smart connect	Axi interconnect		Tạo các cổng slave, master để master và slave truyền, nhận lệnh.
DMA	Slave	AXI Lite AXI Stream	Được Zynq điều khiển init qua AXI Lite. Nhận data output từ IP Softmax qua AXIS.
	Master	AXI Stream	Push data K, Q qua AXIS tới IP Linear.
Linear	Slave	AXI Stream	Nhận K, Q từ DMA.
	Master	AXI Stream	Push Attention score tới Ip Softmax qua AXIS.
Softmax	Slave	AXI Stream	Nhận Attention Score qua AXIS.
	Master	AXI Stream	Push softmax score tới DMA qua AXIS.

BRAM Controller SRC		AXI	Controller ip BRAM SRC, DMA thực hiện MM2S để lấy data.
BRAM SRC			Lưu K, Q data
BRAM Controller DST		AXI	Controller ip BRAM DST, DMA thực hiện S2MM để ghi data.
BRAM DST			Lưu Softmax score data.



Hình 20: Sơ đồ khối Code C

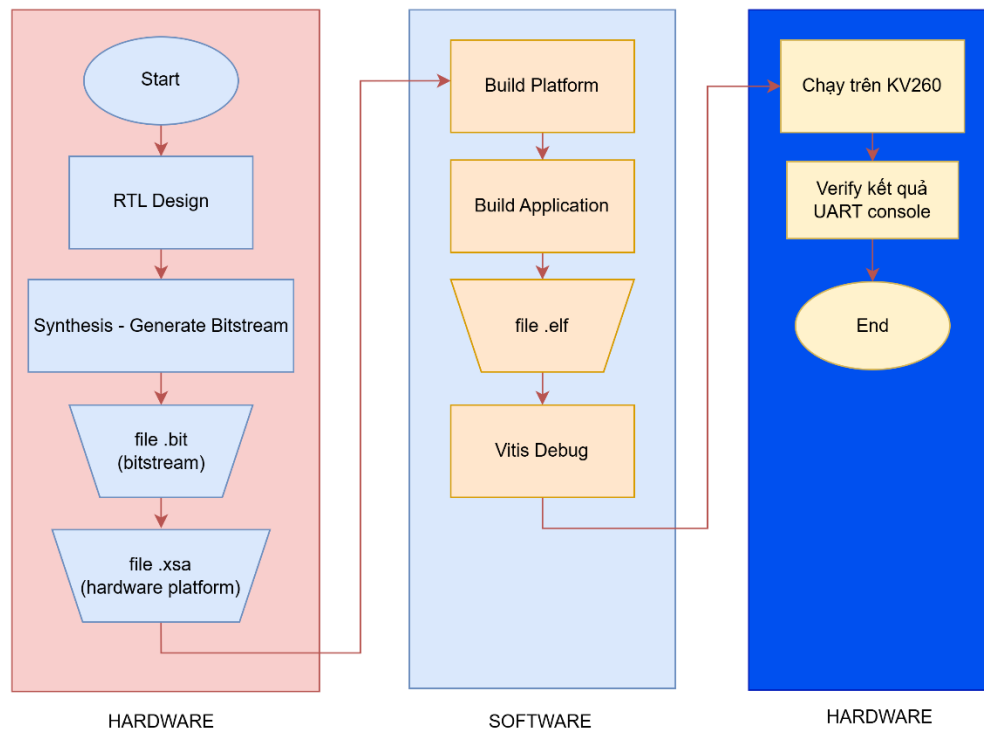
2. Memory mapping

Địa chỉ và kích thước bộ nhớ cho các IP:

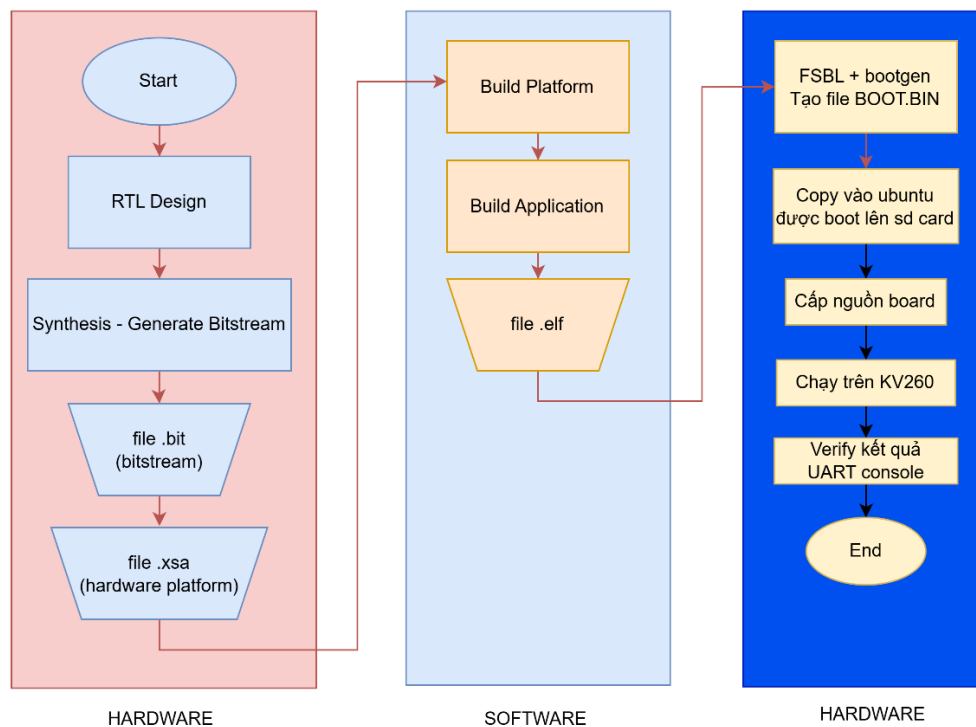
Table 15: Địa chỉ các IP được sử dụng khi thực thi trên KV260

Tên IP	Địa chỉ	Kích thước
DMA	0xA002_0000	64K
Linear	0xA003_0000	64K
Softmax	0xA004_0000	16K
BRAM Controller SRC	0xA000_0000	32K
BRAM Controller DST	0xA001_0000	16K

Sơ đồ khối luồng thực thi được chia ra thành hai phương pháp, phương pháp đầu tiên là boot dự án bằng JTAG, code C thông qua phần mềm Vitis IDE hay còn gọi là Baremetal, phương pháp còn lại là boot bằng linux ubuntu thông qua thẻ microSD:



Hình 21: Sơ đồ khối luồng thực thi bằng Baremetal



Hình 22: Sơ đồ khối luồng thực thi bằng linux thông qua SD card

Do linux quản lý bộ nhớ thông qua cơ chế Bộ nhớ ảo và khối phần cứng MMU, nên mọi chương trình C chỉ được chạy trong vùng ảo. Linux không cho phép chương trình C khai báo con trỏ chỉ thẳng vào địa chỉ vật lý.

Code C sử dụng /dev/mem, đây là file đại diện cho toàn bộ bane đồ không gian bộ nhớ vật lý của chip Zynq, file này sẽ yêu cầu linux nổi địa chỉ vật lý với con trỏ ảo hợp lệ để sử dụng.

3. Thông số thiết lập

Cấu trúc mô phỏng và flow hoàn toàn tương tự so với mô phỏng, chỉ thay đổi cpu thành zynq arm của kit kv260 và thực hiện đồ kit bằng phần mềm Vitis IDE, còn hiệu là bare metal.

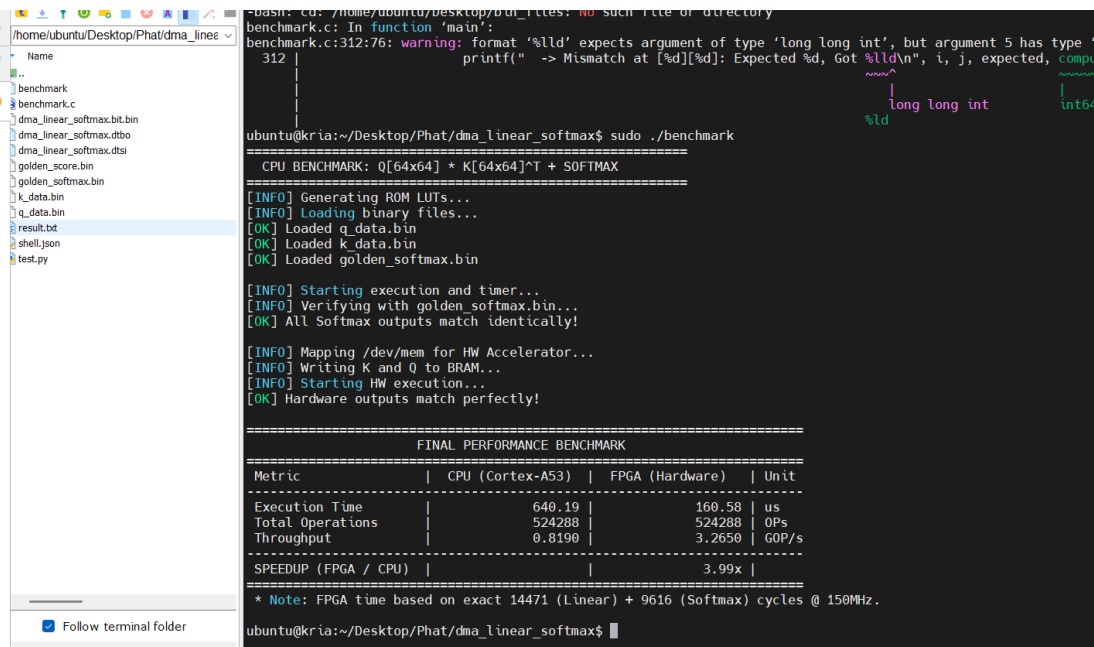
Thực hiện với bộ thông số đầu vào như sau:

Table 16: Thông số chuẩn hóa được sử dụng khi thực thi trên KV260

Parameter	Value
D MODEL	64
SEQ LEN	64
D HEAD	64
N PE	64
DATA WIDTH	16

4. Kết quả thực thi với Ubuntu Linux

Bài nghiên cứu còn thực hiện push dự án lên trên kv260 bằng linux thông qua thẻ microsd, kết quả hoàn toàn đúng so với golden model.



```
bash: cd: /home/ubuntu/Desktop/bui_files: no such file or directory
benchmark.c: In function 'main':
benchmark.c:312:76: warning: format '%lld' expects argument of type 'long long int', but argument 5 has type 'int' [-Wformat]
    printf(" -> Mismatch at [%d][%d]: Expected %d, Got %lld\n", i, j, expected, computed);
                                   ^~~~~~
                                   |
                                   long long int
                                   |
                                   %ld
                                   |
                                   int64_t

ubuntu@kria:~/Desktop/Phat/dma_linear_softmax$ sudo ./benchmark
=====
CPU BENCHMARK: Q[64x64] * K[64x64]^T + SOFTMAX
=====
[INFO] Generating ROM LUTs...
[INFO] Loading binary files...
[OK] Loaded q_data.bin
[OK] Loaded k_data.bin
[OK] Loaded golden_softmax.bin

[INFO] Starting execution and timer...
[INFO] Verifying with golden_softmax.bin...
[OK] All Softmax outputs match identically!

[INFO] Mapping /dev/mem for HW Accelerator...
[INFO] Writing K and Q to BRAM...
[INFO] Starting HW execution...
[OK] Hardware outputs match perfectly!

=====
FINAL PERFORMANCE BENCHMARK
=====
Metric | CPU (Cortex-A53) | FPGA (Hardware) | Unit
-----|-----|-----|-----
Execution Time | 640.19 | 160.58 | us
Total Operations | 524288 | 524288 | OPs
Throughput | 0.8190 | 3.2650 | GOP/s
-----|-----|-----|-----
SPEEDUP (FPGA / CPU) | | 3.99x |
=====
* Note: FPGA time based on exact 14471 (Linear) + 9616 (Softmax) cycles @ 150MHz.
ubuntu@kria:~/Desktop/Phat/dma_linear_softmax$
```

Hình 23: Kết quả thực thi dự án bằng linux với Q[64x64] và K[64x64]

```

* Note: FPGA time based on exact 14471 (Linear) + 9616 (Softmax) cycles @ 150MHz.
ubuntu@kria:~/Desktop/Phat/dma_linear_softmax$ gcc -O3 benchmark.c -o benchmark -lm
sudo ./benchmark
benchmark.c: In function 'main':
benchmark.c:313:76: warning: format '%lld' expects argument of type 'long long int', but
313 | printf(" -> Mismatch at [%d][%d]: Expected %d, Got %lld\n", i, j, result[i][j], golden[i][j])
    |                                     ~~~~~^~~~~~
    |                                     |      |
    |                                     long long int  long long int
    |                                     %ld      %ld

=====
CPU BENCHMARK: Q[128x64] * K[64x64]^T + SOFTMAX
=====
[INFO] Generating ROM LUTs...
[INFO] Loading binary files...
[OK] Loaded q_data.bin
[OK] Loaded k_data.bin
[OK] Loaded golden_softmax.bin

[INFO] Starting execution and timer...
[INFO] Verifying with golden_softmax.bin...
[OK] All Softmax outputs match identically!

[INFO] Mapping /dev/mem for HW Accelerator...
[INFO] Writing K to BRAM (One-time)...
[INFO] Starting HW execution with 2 tile(s)...
[OK] Hardware outputs match perfectly!

=====
FINAL PERFORMANCE BENCHMARK
=====
Metric | CPU (Cortex-A53) | FPGA (Hardware) | Unit
-----|-----|-----|-----
Execution Time | 1321.06 | 320.45 | us
Total Operations | 1048576 | 1048576 | OPs
Throughput | 0.7937 | 3.2722 | GOP/s
-----|-----|-----|-----
SPEEDUP (FPGA / CPU) | | 4.12x |

* Note: FPGA time based on exact 28836 (Linear) + 19232 (Softmax) cycles @ 150MHz.
ubuntu@kria:~/Desktop/Phat/dma_linear_softmax$

```

Hình 24: Kết quả thực thi dự án bằng linux với $Q[128 \times 64]$ và $K[64 \times 64]$

```

* Note: FPGA time based on exact 28836 (Linear) + 19232 (Softmax) cycles @ 150MHz.
ubuntu@kria:~/Desktop/Phat/dma_linear_softmax$ gcc -O3 benchmark.c -o benchmark -lm
sudo ./benchmark
benchmark.c: In function 'main':
benchmark.c:313:76: warning: format '%lld' expects argument of type 'long long int', but
313 | printf(" -> Mismatch at [%d][%d]: Expected %d, Got %lld\n", i, j, result[i][j], golden[i][j])
    |                                     ~~~~~^~~~~~
    |                                     |      |
    |                                     long long int  long long int
    |                                     %ld      %ld

=====
CPU BENCHMARK: Q[256x64] * K[64x64]^T + SOFTMAX
=====
[INFO] Generating ROM LUTs...
[INFO] Loading binary files...
[OK] Loaded q_data.bin
[OK] Loaded k_data.bin
[OK] Loaded golden_softmax.bin

[INFO] Starting execution and timer...
[INFO] Verifying with golden_softmax.bin...
[OK] All Softmax outputs match identically!

[INFO] Mapping /dev/mem for HW Accelerator...
[INFO] Writing K to BRAM (One-time)...
[INFO] Starting HW execution with 4 tile(s)...
[OK] Hardware outputs match perfectly!

=====
FINAL PERFORMANCE BENCHMARK
=====
Metric | CPU (Cortex-A53) | FPGA (Hardware) | Unit
-----|-----|-----|-----
Execution Time | 2580.82 | 640.05 | us
Total Operations | 2097152 | 2097152 | OPs
Throughput | 0.8126 | 3.2766 | GOP/s
-----|-----|-----|-----
SPEEDUP (FPGA / CPU) | | 4.03x |

* Note: FPGA time based on exact 57543 (Linear) + 38464 (Softmax) cycles @ 150MHz.
ubuntu@kria:~/Desktop/Phat/dma_linear_softmax$

```

Hình 25: Kết quả thực thi dự án bằng linux với $Q[256 \times 64]$ và $K[64 \times 64]$

5. So sánh tài nguyên sử dụng

- Dự án thực thi với luồng kiến trúc Streaming IP to IP, sau khi IP Linear tính xong, kết quả được đẩy trực tiếp qua IP Softmax. Dữ liệu được đẩy từ PS qua DDR xuống PL thông qua 1 IP DMA.
- Nghiên cứu của Ye và cộng sự [7] hướng tới việc gia tốc toàn phần chuỗi tính toán của Multi-Head Attention, bao gồm 6 phép nhân ma trận liên tiếp từ khâu tạo Q,K,V, tính điểm Attention, cho đến lớp Linear Projection cuối cùng và mạng FFN. Để xử lý khối lượng tính toán khổng lồ này, kiến trúc [7] đề xuất một mảng tâm thu tái cấu hình động tối ưu hóa tối đa nhân DSP48, kết hợp cùng bộ điều phối dữ liệu hai cấp và kỹ thuật chia ô ma trận để quản lý luồng dữ liệu. Đối với hàm kích hoạt, [7] sử dụng phương pháp xấp xỉ Softmax cơ sở 2 thông qua dịch bit và tra bảng LUT để giảm thiểu độ trễ phần cứng.
- Ở một góc độ tiếp cận khác, kiến trúc FAMOUS do Kabir và cộng sự [8] đề xuất lại đánh đổi việc gia tốc toàn phần để đổi lấy tính linh hoạt cao trong thời gian chạy. Hệ thống [8] được thiết kế dựa trên HLS và chỉ tập trung gia tốc các khâu tính toán cốt lõi bên trong lớp Attention, bỏ qua lớp Linear Projection đầu ra. Thay vì sử dụng bộ điều phối 2 cấp phức tạp như [7], kiến trúc FAMOUS giải quyết giới hạn bộ nhớ BRAM on-chip bằng kỹ thuật phân mảnh theo cột kết hợp với các đường ống xử lý độc lập cho từng Attention Head. Hướng tiếp cận này giúp [8] có thể dễ dàng thay đổi các siêu tham số của mô hình (như kích thước head, chiều dài chuỗi) thông qua phần mềm Host mà không cần tổng hợp lại bitstream phần cứng.

Table 17: Bảng so sánh tài nguyên

Thông số / Chỉ số	[7] (Ye et al.) 16-bit	[8] (Kabir et al. – FAMOUS)	My project (Dự án này)
Tên Kit / FPGA	Xilinx Alveo U250	Xilinx Alveo U55C	Xilinx Kria KV260 (XCK26)
LUT	427 (0,03%)	1 284 782 (98%)	37 423 (31,95%)
FF	369 (0,01%)	661 996 (25%)	87 006 (37,14%)
BRAMs	0,5 (0,02%)	3 148 (78% – 18k)	24 (16,67%)
DSP	–	4 157 (46%)	80 (6,41%)

Dynamic Power (W)	0,007	–	3,031 W
Critical path (ns)	6,443	– (Latency: 0.94 ms)	6,641 ns ($WNS = +0.026$ ns)
Thời gian thực thi (FPGA)	0,642 ms	0,94 ms (8 heads, d = 768)	0,16058 ms (24 087 cycles @ 150 MHz)
Throughput (GOP/s)	1800 GOP/s	328 GOP/s	3.265 GOP/s
Speedup (FPGA / CPU)	51.3× (vs Xeon CPU)	3.28× (vs Intel Xeon 5220R)	3.99× (vs ARM Cortex-A53)

Tính throughput của dự án FPGA với công thức

$$\begin{aligned}
 \text{throughput} \left(\frac{GOP}{s} \right) &= \frac{\text{Total operation}}{\text{Executive time (s)}} = \frac{2 * SEQ\ LEN * D\ HEAD * D\ MODEL}{160,58\ (us)} \\
 &= \frac{2 * 64 * 64 * 64}{160,58\ (us)} = 3,26\ GOP/s
 \end{aligned}$$

Tính throughput của CPU:

$$\begin{aligned}
 \text{throughput} \left(\frac{GOP}{s} \right) &= \frac{\text{Total operation}}{\text{Executive time (s)}} = \frac{2 * SEQ\ LEN * D\ HEAD * D\ MODEL}{640,19\ (us)} \\
 &= \frac{2 * 64 * 64 * 64}{640,19\ (us)} = 3,26\ GOP/s
 \end{aligned}$$

V. TÀI LIỆU THAM KHẢO

- [1] ARM Limited, "AMBA AXI4-Lite Interface Specification," ARM IHI0022L, 2022.
- [2] ARM Limited, "AMBA AXI4-Stream Protocol Specification," ARM IHI0051B, 2021.
- [3] Xilinx Inc., "AXI DMA v7.1 Product Guide," PG021, AMD/Xilinx, 2022.
- [4] Xilinx Inc., "AXI Reference Guide," UG761, AMD/Xilinx, 2012.
- [5] R. Sass and A. G. Schmidt, The Zynq Book: Embedded Processing with the ARM Cortex-A9 on the Xilinx Zynq-7000 All Programmable SoC, Strathclyde Academic Media, 2014.
- [6] Xilinx Inc., "Zynq-7000 SoC: Embedded Design Tutorial, IP Creation with Vivado," AMD/Xilinx, 2016.

[7] Wenhua Ye, Xu Zhou, Joey Tianyi Zhou, Cen Chen, Kenli Li, “Accelerating attention mechanism on FPGAs based on efficient reconfigurable systolic array” 2022.

[8] Ehsan Kabir, Md. Arafat Kabir, Austin R.J. Downey, Jason D. Bakos, David Andrews, Miaoqing Huang, “FAMOUS: Flexible Accelerator for the Attention Mechanism of Transformer on UltraScale+ FPGAs” 2024.

VII. PHỤ LỤC

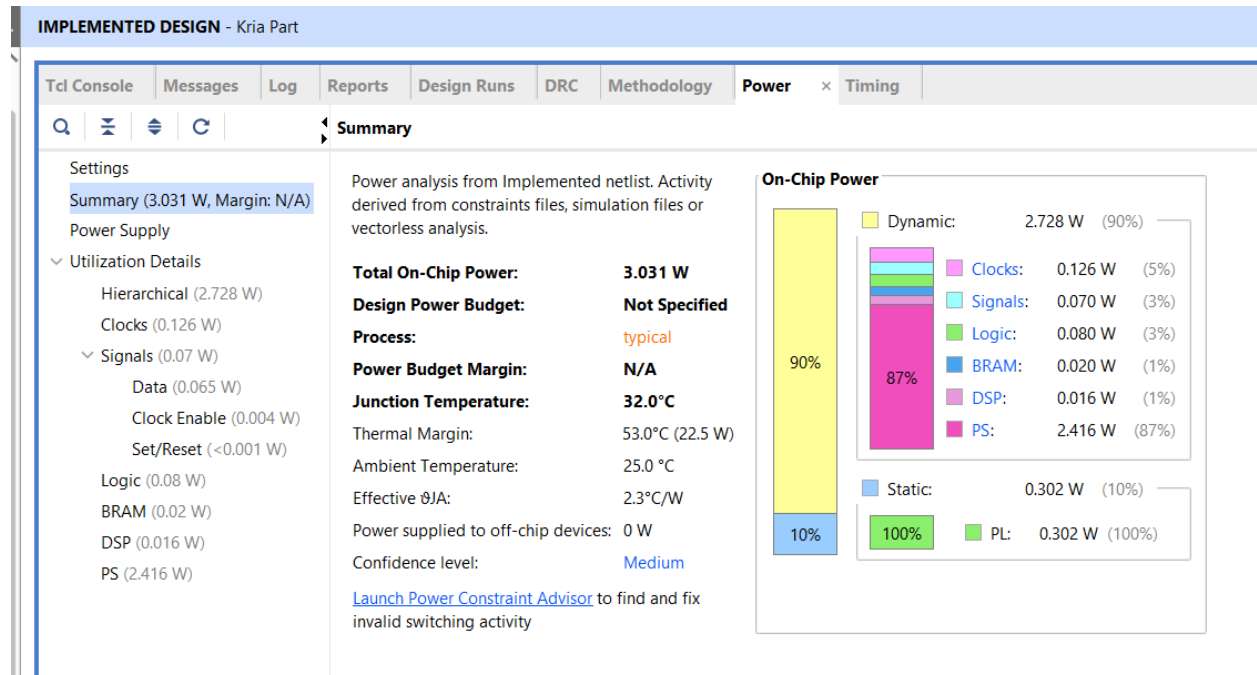
1. Phụ lục A – Mã nguồn dự án

Dự án được lưu trữ tại github: [PHATVP22VT \(Arthur-Artoria\)](#)

2. Phụ lục B – Report Utilization

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS): 0.026 ns		Worst Hold Slack (WHS): 0.010 ns		Worst Pulse Width Slack (WPWS): 2.000 ns	
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns		Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	
Total Number of Endpoints: 196070		Total Number of Endpoints: 196070		Total Number of Endpoints: 88766	
All user specified timing constraints are met.					

Hình 26: Design Timing Summary



Hình 27: Power summary

IMPLEMENTED DESIGN - Kria Part

Tcl ConsoleMessagesLogReportsDesign Runs × DRCMethodologyPowerTiming

Q≡⚙️⏮️⏪⏩⏭️+

?

⏮️⏭️⏩⏪⏫

	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF	BRAM	URAM	DSP	Start	Elapsed	Run Strategy
complete!												39838	87056	24	0	80	7/29/26, 2:13 PM	00:17:44	Vivado Synth
complete!	0.026	0.000	0.010	0.000		0.000	3.031	0	305 Warn			37423	87006	24	0	80	7/29/26, 2:30 PM	00:26:26	Vivado Imple

Hình 28: Design Runs Summary